

Diego Jacobo Esparza
Leonardo Daniel Pérez Velázquez

Computabilidad en autómatas celulares complejos

– Trabajo Terminal –

18 de mayo de 2026

Escuela Superior de Cómputo
Instituto Politécnico Nacional

Advertencia

“Este informe contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional a partir de datos y documentos con derecho de propiedad y por lo tanto su uso queda restringido a las aplicaciones que explícitamente se convengan.”

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Avenida Juan de Dios Bátiz s/n.
Teléfono: 57 296 000, extensión 52000.

Prólogo

El estudio de los autómatas celulares es una de las áreas más interesantes dentro de la teoría de la computación y de los sistemas dinámicos discretos. A pesar de que parte de reglas relativamente simples, los modelos han demostrado ser capaces de producir comportamientos complejos, llegando a representar procesos que se relacionan con computación universal. Debido a esto, los autómatas celulares continúan siendo objeto de investigación en diferentes ramas de matemáticas y la computación.

El presente Trabajo Terminal toca temas justamente en esa línea de investigación, enfocándose en el análisis de la computabilidad en autómatas celulares. A lo largo del presente documento, los autores desarrollan una aproximación fundamentada en teoría sobre topología discreta, dinámica de sistemas y teoría de autómatas, lo cual permite interpretar fenómenos emergentes dentro de espacios celulares bidimensionales. Uno de los aspectos a destacar en este trabajo es el esfuerzo por mantener una estructura rigurosa en la exposición de las ideas. El documento no se limita únicamente a describir comportamientos que se observaron en la experimentación, sino que intenta justificar las propiedades computacionales asociadas a reglas complejas como *Spiral* y *Beehive*. Este enfoque requiere una comprensión sólida de los fundamentos teóricos involucrados. Un punto importante es la realización establecida entre las colisiones de estructuras y la posibilidad de sintetizar operaciones lógicas. La idea de usar gliders y dinámicas emergentes como mecanismos de procesamiento de información representa una aproximación distinta al paradigma clásico de Von Neumann. La dirección general de la investigación se encuentra claramente definida. También es evidente el esfuerzo que se puso para mantener la coherencia entre la parte teórica y la implementación propuesta. Validar los modelos mediante simulaciones aporta un componente práctico relevante que permite trasladar conceptos teóricos a un entorno “tangible”. El trabajo aquí presentado constituye una aportación valiosa al campo de estudio al que se enfoca, que puede servir como base para futuras investigaciones relacionadas con autómatas celulares y modelos alternativos de computación.

Mayo de 2026

Juan Alberto Rugerio Arceo

Prefacio

El presente documento expone los fundamentos teóricos, matemáticos y algorítmicos que sustentan la investigación titulada “Computabilidad en autómatas celulares complejos”. El propósito central de este trabajo es formalizar y demostrar la capacidad de ciertos sistemas dinámicos discretos, específicamente bajo las reglas de transición *Spiral* y *Beehive*, para ejecutar procesos de cómputo universal a través de la colisión determinista de configuraciones móviles o *gliders*.

A lo largo del desarrollo de la ingeniería y las ciencias de la computación, es recurrente observar cómo problemas que inicialmente parecen abstractos o puramente teóricos exigen soluciones arquitectónicas y algorítmicas que permiten comprender el manejo de la información a bajo nivel. Bajo esta premisa, el trabajo se encuentra estructurado en una progresión axiomática que guía al lector desde las definiciones más fundamentales hasta el análisis de complejidad algorítmica.

En el Capítulo 1, se delimita formalmente el objeto de estudio, estableciendo los criterios analíticos y sintéticos necesarios para evaluar la emergencia computacional en los modelos seleccionados. Posteriormente, el Capítulo 2 consolida el marco conjuntista y topológico, abordando el concepto crítico de la irreversibilidad en la evolución del autómata; se demuestra cómo la aniquilación de información y la generación de estados Jardín del Edén son requisitos axiomáticos para emular compuertas lógicas tradicionales.

El diseño de un motor matemático computacional exige una comprensión profunda de las estructuras espaciales subyacentes. Por ello, el Capítulo 3 detalla la geometría analítica de la teselación hexagonal y justifica las decisiones de implementación, comprobando teóricamente que la complejidad temporal de evaluación y la actualización mediante *double buffering* mantienen una cota lineal $O(n)$. Esta base permite formalizar, en el Capítulo 4, el mapeo topológico esencial del proyecto: la transformación biyectiva de una matriz hexagonal a una subred ortogonal de memoria. Mediante el modelo algebraico de transformación definido en la Ecuación (4.15) y la preservación de la métrica de distancia de la Ecuación (4.5), se garantiza la invariancia espacial, ilustrada estructuralmente en la Figura 4.4.

Finalmente, el Capítulo 5 aborda la fenomenología de estas partículas en movimiento, culminando con la evaluación algorítmica de sus interacciones. Se

formaliza que la sincronización determinista de *gliders* para el diseño de circuitos lógicos recae en la clase de complejidad NP, validando la necesidad de los métodos de búsqueda iterativa descritos por la Ecuación (5.3).

Este documento está dirigido a estudiantes de ingeniería, matemáticos e investigadores interesados en la computación no convencional, la teoría de autómatas y el análisis de sistemas complejos. La comprensión competente de estos modelos lógicos y espaciales ofrece una perspectiva rigurosa sobre cómo reglas simples, ejecutadas masivamente en paralelo, pueden producir resultados coherentes, estables y computacionalmente universales.

Ciudad de México, México
Estado de México, México
Mayo de 2026

Esparza Jacobo Diego
Pérez Velázquez Leonardo Daniel

Índice

Prólogo	vii
Prefacio	ix
Parte I Fundamentos axiomáticos y fenomenología de sistemas dinámicos discretos	
1 Preliminares y planteamiento del problema computacional	3
1.1 Contexto topológico y fenomenológico	3
1.2 Definición formal del objeto de estudio	4
1.3 Delimitación axiomática e hipótesis fundamental	5
1.3.1 Hipótesis	5
1.4 Objetivos analíticos y sintéticos	5
1.4.1 Criterios analíticos	5
1.4.2 Criterios sintéticos	6
2 Teoría formal de autómatas celulares y espacios discretos	7
2.1 Fundamentación conjuntista y topológica del espacio celular	7
2.1.1 Definición del espacio y la unidad celular	7
2.1.2 Definición axiomática de autómata celular	8
2.1.3 Alfabetos finitos y el espacio de estados	8
2.1.4 Formalización métrica de vecindades acotadas	9
2.1.5 El autómata celular hexagonal	9
2.2 Funciones de transición y evolución espacio-temporal	10
2.2.1 La función de transición local	11
2.2.2 Configuraciones globales y el mapeo paralelo	11
2.2.3 Discretización del tiempo y trayectoria evolutiva	11
2.3 Dinámica global y clasificación cualitativa del espacio de fases	12
2.3.1 Atractores espaciales y estabilidad asintótica	13
2.3.2 Formalización de la clasificación de Wolfram	13
2.4 Teoría de la computabilidad sobre sistemas dinámicos discretos	15

2.4.1	Equivalencia algorítmica y Máquinas de Turing	15
2.4.2	Emergencia de operadores lógicos y universalidad computacional	15
2.5	Reversibilidad topológica y dinámica de la información	16
2.5.1	Inyectividad, suprayectividad y teoremas fundamentales	17
2.5.2	Entropía discreta y estados Jardín del Edén	17

Parte II Morfología, geometría estructural y equivalencia topológica

3	Geometría computacional y formación algebraica para la representación espacial	21
3.1	Estructuras algebraicas para retículas espaciales	21
3.1.1	Definición axiomática del espacio bidimensional discreto	21
3.1.2	Topología métrica de la red celular	22
3.2	Geometría analítica de la tesela fundamental hexagonal	23
3.2.1	Parametrización geométrica de la celda unitaria en $\mathbb{R}^2$	24
3.2.2	Formulación del dominio poligonal y ecuaciones de contorno	24
3.3	Transformaciones afines y mapeo de coordenadas globales	25
3.3.1	Operadores matriciales para la traslación espacial	26
3.3.2	Sistemas paramétricos de coordenadas compensadas (<i>Offset Coordinates</i>)	27
3.4	Topologías compactas y axiomatización de fronteras	28
3.4.1	Construcción algebraica del espacio cociente toroidal	28
3.4.2	Ecuaciones de transición para condiciones de contorno periódicas	30
3.5	Complejidad computacional del mapeo topológico y renderizado	30
3.5.1	Mapeo espacial inverso y localización de puntos	31
3.5.2	Análisis espacial y temporal de la actualización topológica	31
4	Isomorfismos y transformaciones topológicas entre espacios celulares	33
4.1	Axiomatización geométrica de las teselaciones hexagonales y métricas planares	33
4.1.1	Representación paramétrica de la celda hexagonal regular	33
4.1.2	Sistemas de coordenadas afines para mallas de grado seis	34
4.1.3	Definición formal de vecindad de adyacencia planar $\mathcal{N}_H$	34
4.2	Homomorfismos espaciales y condiciones de equivalencia reticular	35
4.2.1	Isomorfismos sobre grafos reticulares infinitos	36
4.2.2	Teorema de recubrimiento y descomposición celular ortogonal	36
4.3	Modelado analítico de la transformación afin: De morfología hexagonal a retícula $\mathbb{Z}^2$	37
4.3.1	Ecuaciones de transformación de base espacial	38
4.3.2	Formalización de la correspondencia submatricial 2×2	38
4.3.3	Tratamiento algebraico del desfase axial (Zig-Zag Offset)	39
4.4	Mapeo topológico de vecindades y compensación dimensional	40

4.4.1	Construcción algebraica de vecindades extendidas sobre $\mathbb{Z}^2$	41
4.4.2	Mapeo direccional y compensación de métricas de adyacencia	41
4.5	Teoremas de invariancia dinámica y preservación de la computabilidad	43
4.5.1	Invarianza de la función de transición local δ bajo isomorfismos de red	43
4.5.2	Teorema de equivalencia computacional y conservación del espacio de fases	44

Parte III Fenómenos emergentes y lógica de colisiones

5	Dinámica de partículas y computación por interacciones locales	49
5.1	Fundamentos algorítmicos y fenomenología de subconfiguraciones móviles	49
5.1.1	Definición conjuntista y caracterización algebraica de <i>gliders</i>	49
5.1.2	Diseño de algoritmos de detección y rastreo de trayectorias en retículas discretas	50
5.2	Caracterización algebraica y mecánica algorítmica de la dinámica Spiral	52
5.2.1	Formalización de la función de transición Spiral: Espacio de estados y dinámica de fases	52
5.2.2	Algoritmo propuesto para la optimización de la cinemática de propagación Spiral	53
5.3	Topología de atractores y evaluación algorítmica en la dinámica Beehive	55
5.3.1	Estudio analítico de ciclos límite y osciladores bajo la función de transición Beehive	55
5.3.2	Algoritmos de resolución para aglomeraciones densas y predicción de auto-organización	56
5.4	Diseño computacional para la síntesis de operadores booleanos mediante colisiones	58
5.4.1	Isomorfismo algorítmico entre la topología de intersecciones y las compuertas lógicas universales	58
5.4.2	Algoritmo heurístico propuesto para la programación espacial de circuitos emergentes	59
5.4.3	Tratabilidad computacional y cotas de complejidad en la sincronización de señales de control	60
6	Estructuración lógica y arquitectura computacional del motor de simulación	63
6.1	Especificación formal de requerimientos del sistema	63
6.1.1	Taxonomía de requerimientos funcionales	63
6.1.2	Restricciones y atributos de calidad (Requerimientos no funcionales)	66

6.2	Topología arquitectónica general y abstracción en capas	67
6.2.1	Diagrama de la arquitectura del sistema	67
6.2.2	Taxonomía de las capas estructurales	68
6.3	Núcleo de simulación: Lógica fundamental y representación de estados	70
6.3.1	Encapsulamiento del sistema dinámico	70
6.3.2	Operaciones estructurales y evolutivas	71
6.3.3	Estructuras de almacenamiento discreto y optimización espacial	72
6.4	Control de la dinámica temporal del autómata	72
6.4.1	Delimitación entre el dominio lógico y el temporal	73
6.4.2	Administración de estados de ejecución y sincronización ...	74
6.5	Aislamiento espacial y multiplicidad de entornos (Sistema Multi-Viewport)	75
6.5.1	Instanciación independiente de contextos evolutivos	76
6.5.2	Composición y orquestación del entorno aislado	76
6.6	Mapeo geométrico y representación visual discreta	77
6.6.1	Desacoplamiento de la geometría y el estado lógico	78
6.6.2	Implementaciones de transformación espacial	79
6.7	Orquestación global e interacción asíncrona del sistema	80
6.7.1	Coordinación del ciclo principal y sincronización de estados	80
6.7.2	Organización modular de la interfaz de usuario	81
6.8	Consecuencias arquitectónicas y proyecciones futuras	82
6.8.1	Evaluación de la modularidad y robustez del sistema	83
6.8.2	Líneas de refinamiento y herramientas analíticas desacopladas	83
6.8.3	Optimización asintótica y arquitecturas orientadas a datos ..	84
Referencias		85

Lista de figuras

1.1	Representación geométrica de una vecindad bidimensional hexagonal y el flujo de información determinista durante la evaluación temporal. El estado de la célula central s_c^{t+1} emerge como el mapeo estricto de los estados adyacentes s_i^t a través de la función de transición local Φ	4
2.1	Comparativa geométrica topológica entre el espacio de cuadrícula $\mathbb{Z}^2$ asimétrica y la teselación celular hexagonal isótropa. Se detallan los vectores de desplazamiento $\mathbf{v}_i$ que definen la vecindad adyacente $\mathcal{N}_{hex}$	10
2.2	Representación del mapeo de estados locales a la evolución de la configuración global del sistema.	12
2.3	Representación esquemática del espacio de fases discreto. Se observan los estados transitorios que conforman las cuencas de atracción convergiendo en estados estables y ciclos límite.	14
2.4	Isomorfismo entre el modelo de Máquina de Turing y la evolución dinámica de un autómata celular bidimensional.	16
2.5	Representación topológica de la irreversibilidad en el espacio de estados. La convergencia de trayectorias ilustra la pérdida de información, mientras que los nodos sin preimágenes denotan estados Jardín del Edén.	18
3.1	Representación geométrica de las vecindades inducidas por distintas métricas en $\mathbb{Z}^2$. Se observa cómo la elección de la distancia determina la morfología de la interacción local.	23
3.2	Geometría analítica de la tesela hexagonal unitaria. Se destacan las relaciones trigonométricas entre el circunradio y la apotema.	25
3.3	Representación del dominio $\mathcal{H}$ como intersección de semiplanos. Las regiones sombreadas indican el cumplimiento de las restricciones lineales.	26

3.4	Modelado geométrico del sistema de coordenadas compensadas <i>odd-r</i> . Se ilustra cómo la paridad de la fila j determina la traslación sobre el eje de las abscisas, emulando un empaquetamiento óptimo continuo a partir de índices discretos.	28
3.5	Representación del espacio cociente $\mathbb{T}^2$. Las fronteras opuestas se identifican algebraicamente para construir una topología continua libre de condiciones de borde.	29
3.6	Esquematización del mapeo algebraico inverso. La transición del dominio continuo de pantalla hacia el arreglo de memoria discreto. ...	32
4.1	Representación de la vecindad $\mathcal{N}_H$ y el sistema de coordenadas cúbicas subyacente.	35
4.2	Equivalencia topológica: Mapeo isomórfico suprayectivo entre una celda hexagonal con grado de adyacencia 6 y una partición cuadrangular de 2×2	37
4.3	Representación gráfica de la discontinuidad espacial en un mapeo de coordenadas <i>zig-zag</i> . Se denota el desfase provocado por la paridad de la columna.	40
4.4	Distribución de los seis ejes direccionales de interacción sobre el perímetro expuesto de la subred funcional de adyacencia $\mathcal{T}_B$	42
4.5	Representación visual del isomorfismo entre la evolución temporal y la transformación espacial.	45
5.1	Representación gráfica del rastreo de ventana deslizante evaluando una subconfiguración patrón $\mathbf{P}$ sobre la retícula bidimensional transformada $\mathbf{X}$	51
5.2	Evolución temporal de la dinámica Spiral. Se observa la formación de frentes de onda y la estela de estados de decaimiento que orientan la propagación.	54
5.3	Esquema lógico del particionamiento espacial. La omisión algorítmica de los macro-bloques en reposo reduce el número de operaciones de lectura/escritura en la memoria.	57
5.4	Representación del retroceso cinemático para la sincronización de colisiones. El algoritmo determina las semillas iniciales asegurando que las fases morfológicas coincidan en el punto de intersección (i_c, j_c)	60
6.1	Mapa conceptual de la interdependencia entre los requerimientos funcionales de simulación y las restricciones de diseño arquitectónico.	68
6.2	Diagrama estructural de la arquitectura del sistema. Se ilustra la jerarquía de capas y el aislamiento del núcleo de simulación respecto a la interfaz de usuario.	69
6.3	Diagrama de encapsulamiento del núcleo. Se observa la agregación de la lógica de reglas y el almacenamiento optimizado dentro de la entidad Automaton.	73

6.4	Máquina de estados finitos que rige la clase Simulation, ilustrando las transiciones entre la pausa, la ejecución continua y el avance paso a paso controlado.....	75
6.5	Estructura interna del sistema Multi-Viewport. Se muestra el encapsulamiento de los motores lógicos y operacionales bajo un mismo contexto de representación visual.	78
6.6	Arquitectura del subsistema de visualización. Se ilustra cómo las especializaciones de renderizado encapsulan la lógica de transformación geométrica sin afectar al núcleo de simulación.	80
6.7	Modelo estructural de la interfaz de usuario. Se ilustra la independencia de los paneles analíticos respecto al ciclo de simulación central.	82

Lista de tablas

6.1	Especificación del requerimiento funcional para la gestión de entornos múltiples.	64
6.2	Especificación del requerimiento funcional para la flexibilidad paramétrica.	64
6.3	Especificación del requerimiento funcional para la edición de mallas.	64
6.4	Especificación del requerimiento funcional para la fidelidad topológica.	64
6.5	Especificación del requerimiento funcional para el análisis temporal discreto.	65
6.6	Especificación del requerimiento funcional para el control de ejecución.	65
6.7	Especificación del requerimiento funcional para persistencia de configuraciones.	65
6.8	Especificación del requerimiento funcional para recuperación experimental.	65
6.9	Especificación del requerimiento funcional para seguimiento de estructuras móviles.	65
6.10	Especificación del requerimiento funcional para detección de periodicidad.	66
6.11	Especificación del requerimiento no funcional para la eficiencia de almacenamiento.	66
6.12	Especificación del requerimiento no funcional para la mantenibilidad.	66
6.13	Especificación del requerimiento no funcional para la integridad lógica.	66
6.14	Especificación del requerimiento no funcional para escalabilidad espacial.	67
6.15	Especificación del requerimiento no funcional para consistencia formal.	67
6.16	Comparativa de eficiencia espacial en la representación del estado celular.	72

Siglas

A continuación se presenta el listado de las siglas y acrónimos empleados a lo largo del presente documento, así como la definición formal de los conceptos subyacentes.

AC	Autómata Celular. Sistema dinámico y discreto en el espacio, el tiempo y el estado, cuya evolución es regida por reglas de transición locales.
AND	Compuerta lógica de conjunción, definida por retornar un estado afirmativo si y solo si la totalidad de sus operandos son afirmativos.
CPU	<i>Central Processing Unit</i> . Unidad Central de Procesamiento encargada de la ejecución secuencial de las instrucciones lógicas y aritméticas en la arquitectura subyacente.
ECS	<i>Entity Component System</i> . Patrón arquitectónico de diseño de software que estructura el sistema mediante la desvinculación estricta de datos y operaciones, optimizando la iteración sobre la memoria caché.
GE	<i>Garden of Eden</i> . Configuración del espacio celular o estado de tipo Jardín del Edén, caracterizada axiomáticamente por la inexistencia de una preimagen en la función de transición global.
ID	Identificador. Cadena alfanumérica o numérica unívoca asignada para distinguir de manera rigurosa una entidad, componente o requerimiento.
JSON	<i>JavaScript Object Notation</i> . Formato estándar y ligero de intercambio de información, empleado para la serialización y persistencia de las configuraciones de la malla discreta.
MB	Megabyte. Unidad de medida estándar para el almacenamiento de información en sistemas digitales, equivalente a 2^{20} bytes.
MT	Máquina de Turing. Modelo matemático abstracto de computación, propuesto como marco axiomático para delimitar la resolubilidad y complejidad algorítmica de los lenguajes formales.
NAND	<i>Not AND</i> . Compuerta lógica que opera como la negación de una conjunción, demostrada formalmente como un operador computacional universal.
NOT	Compuerta lógica de negación, la cual invierte estrictamente el valor de verdad de una proposición dada.

NP	<i>Nondeterministic Polynomial time</i> . Clase de complejidad computacional conformada por el conjunto de problemas de decisión cuyas soluciones pueden ser verificadas por una máquina determinista en tiempo polinomial.
OR	Compuerta lógica de disyunción inclusiva, definida por arrojar un valor afirmativo si al menos uno de sus operandos es afirmativo.
RF	Requerimiento Funcional. Declaración explícita de un comportamiento, interacción o procesamiento que el sistema computacional debe ejecutar intrínsecamente por diseño.
RNF	Requerimiento No Funcional. Restricción o atributo de calidad del sistema que delimita los criterios operativos de rendimiento, sincronismo o mantenibilidad estructural.
SAT	<i>Boolean Satisfiability Problem</i> . Problema algorítmico centrado en determinar la existencia de una asignación de variables que satisfaga íntegramente una fórmula booleana dada.
SIMD	<i>Single Instruction, Multiple Data</i> . Arquitectura de procesamiento a nivel de registro que permite la aplicación concurrente de una misma operación sobre vectores de múltiples operandos.
TPS	<i>Ticks Per Second</i> . Métrica empleada para cuantificar la tasa de convergencia temporal, definiendo el número de transiciones globales ejecutadas por el autómata celular en el transcurso de un segundo físico.
UML	<i>Unified Modeling Language</i> . Lenguaje unificado de modelado de propósito general en la ingeniería de software, empleado para la especificación visual y rigurosa de la arquitectura y la dinámica del sistema.

Parte I
Fundamentos axiomáticos y fenomenología
de sistemas dinámicos discretos

Esta primera parte del documento establece las bases teóricas y axiomáticas para el estudio de la computabilidad en autómatas celulares complejos. Se aborda la computación no convencional desde la perspectiva de los sistemas dinámicos discretos, definiendo las propiedades matemáticas que permiten emplear dinámicas naturales para el procesamiento de información. El objetivo principal de esta sección no es únicamente introducir conceptos, sino estructurar el problema desde una perspectiva formal antes de proceder a su implementación algorítmica.

En el Capítulo 1, se define formalmente el objeto de estudio y se plantea el problema computacional. Se establece el contexto fenomenológico en la frontera entre la complejidad topológica, la reversibilidad y la computabilidad universal. Asimismo, se delimitan las hipótesis sobre la emergencia en reglas de evolución específicas, como *Spiral* y *Beehive*, y se presentan los criterios de demostración analíticos y sintéticos (Sección 1.4) que regirán la validación del modelo propuesto.

Posteriormente, en el Capítulo 2, se construye la teoría matemática de los autómatas celulares. A partir de fundamentos de teoría de conjuntos y topología discreta, se axiomatizan los componentes fundamentales del sistema: la retícula espacial, las vecindades acotadas y las funciones de transición. Se analiza la dinámica global del espacio de configuraciones y se formaliza el papel de la irreversibilidad. Este último concepto es crucial, ya que la disipación de estados es un requerimiento matemático indispensable para el diseño de operaciones lógicas deterministas mediante colisiones.

En conjunto, los axiomas y definiciones presentados en esta parte establecen el terreno lógico estrictamente necesario para el desarrollo del modelo. Esta fundamentación es la base sobre la cual se construirá la representación analítica del Capítulo 3, la demostración de transformaciones y mapeos del Capítulo 4, y el análisis cinemático y de complejidad algorítmica del Capítulo 5.

Capítulo 1

Preliminares y planteamiento del problema computacional

Resumen Este capítulo establece los fundamentos axiomáticos y el marco fenomenológico de la investigación sobre la computabilidad en autómatas celulares complejos. Se aborda la computación no convencional desde la perspectiva de los sistemas dinámicos discretos, definiendo formalmente el objeto de estudio en la intersección de la topología discreta, la reversibilidad y la computabilidad universal.

Asimismo, se delimitan las propiedades de emergencia esperadas en reglas específicas de evolución, como *Spiral* y *Beehive*, y se plantean los objetivos analíticos y sintéticos que regirán la demostración formal del modelo matemático y computacional propuesto.

1.1 Contexto topológico y fenomenológico

La teoría de autómatas celulares proporciona un marco matemático robusto para el estudio de sistemas dinámicos discretos donde la complejidad emerge de interacciones estrictamente locales. Históricamente concebidos por von Neumann y Ulam como modelos abstractos de autorreplicación biológica [1], estos sistemas han evolucionado hacia un paradigma fundamental dentro de la computación no convencional.

En este contexto, la computación trasciende el modelo clásico de la Máquina de Turing de estados centralizados, distribuyendo la capacidad de procesamiento sobre una malla espacial (topología). Cada unidad del espacio, denominada **célula**, opera de manera síncrona y paralela, evaluando una función de transición compartida a lo largo de todo el arreglo. Es posible observar que, bajo ciertas configuraciones y reglas de evolución, la dinámica global del autómata pierde su aparente simplicidad para albergar el tránsito y la manipulación de información, dando lugar a fenómenos de computación universal [2].

1.2 Definición formal del objeto de estudio

Para analizar la computabilidad dentro de un espacio bidimensional, es imperativo establecer las bases que rigen su dinámica. Se define un autómata celular complejo como un sistema cuya evolución a lo largo del tiempo discreto exhibe estructuras coherentes y patrones no triviales, situándose en la frontera entre el orden periódico y el caos puro.

El objeto de estudio central de este trabajo se focaliza en la delimitación de una frontera de transición computacional. Dicha frontera se encuentra determinada por tres propiedades fundamentales:

1. **Complejidad topológica:** La capacidad del espacio celular para soportar la existencia de *gliders* (partículas o estructuras móviles) que se desplazan de manera periódica sin perder su integridad morfológica [3].
2. **Reversibilidad frente a irreversibilidad:** La evaluación del sistema respecto a la conservación de la información. Mientras que un sistema reversible asegura una biyección en sus mapeos de estados, las dinámicas complejas a menudo requieren de propiedades irreversibles para colapsar información, aniquilar estados u operar como compuertas lógicas universales [4].
3. **Computabilidad universal:** El grado en el cual un conjunto de colisiones predecibles puede emular funcionalmente a cualquier función booleana discreta y, por extensión, operaciones algorítmicas completas.

Para fines de ilustración fenomenológica, se modela el vecindario y su transición de estados geométricos en el siguiente diagrama.

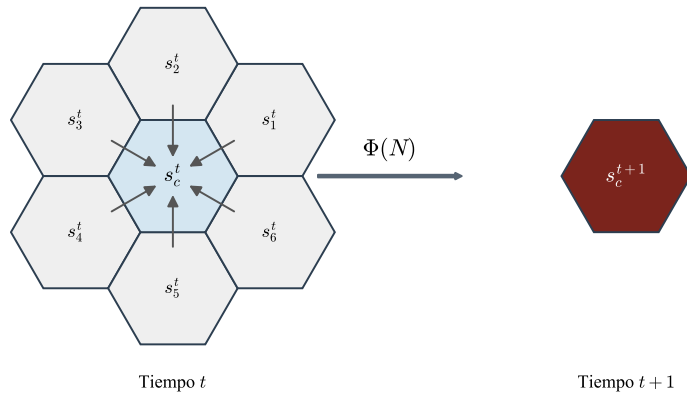


Fig. 1.1 Representación geométrica de una vecindad bidimensional hexagonal y el flujo de información determinista durante la evaluación temporal. El estado de la célula central s_c^{t+1} emerge como el mapeo estricto de los estados adyacentes s_i^t a través de la función de transición local Φ .

1.3 Delimitación axiomática e hipótesis fundamental

La aproximación teórica abordada en el presente documento requiere la fijación de reglas de transición particulares que restrinjan el comportamiento global del sistema a un universo analizable. Se seleccionan dinámicas asociadas a autómatas celulares con alfabetos finitos que exhiben comportamiento complejo, específicamente enfocando el análisis en dominios matemáticos equivalentes a las reglas conocidas empíricamente como *Spiral* y *Beehive*.

1.3.1 Hipótesis

La hipótesis fundamental del Trabajo Terminal establece que *es matemáticamente posible modelar y mapear funciones computacionales complejas utilizando las propiedades emergentes de colisión y aniquilación de estructuras viajeras (gliders) generadas por las reglas de transición no reversibles de los sistemas Spiral y Beehive*.

De lo anterior se axiomatiza que, si es posible predecir el vector de desplazamiento de la partícula y su resultante post-colisión, entonces el conjunto de reglas del autómata celular es sintéticamente equipotente a un circuito combinacional discreto.

1.4 Objetivos analíticos y sintéticos

Para demostrar empírica y matemáticamente la validez de la hipótesis propuesta, la investigación se divide en objetivos de análisis del sistema y objetivos de síntesis lógica:

1.4.1 Criterios analíticos

- **Caracterización del espacio de estados:** Demostrar algebraicamente las propiedades de estabilidad y periodo de las estructuras emergentes en los modelos seleccionados.
- **Isomorfismo espacial:** Establecer las equivalencias topológicas necesarias para garantizar que una topología hexagonal pueda ser correctamente mapeada y simulada dentro de una matriz ortogonal de $\mathbb{Z}^2$ sin pérdida de propiedades computacionales.

1.4.2 Criterios sintéticos

- **Lógica de colisiones:** Documentar el catálogo formal de reacciones generadas tras la intersección de dos o más *gliders* en trayectorias calculadas.
- **Construcción de operadores:** Ensamblar teóricamente, y posteriormente verificar a través del simulador a desarrollar en C/C++, la construcción de compuertas lógicas universales (e.g., AND, NOT) sustentadas exclusivamente por la colisión determinista de partículas dentro del arreglo celular.

De esta manera, el presente proyecto de titulación busca sentar una base matemática formal que valide el uso de dinámicas naturales discretas como mecanismos viables para la ingeniería del procesamiento de información.

Capítulo 2

Teoría formal de autómatas celulares y espacios discretos

Resumen Este capítulo establece el marco matemático fundamental sobre el cual se sustenta la dinámica de los autómatas celulares. A partir de conceptos de la teoría de conjuntos, topología discreta y álgebra geométrica, se construyen las definiciones axiomáticas del espacio celular, los alfabetos finitos, las vecindades y las funciones de transición. Posteriormente, se abordan las propiedades topológicas que permiten el modelado de la computación sobre arquitecturas no convencionales, dotando de formalismo a las simulaciones de sistemas complejos.

2.1 Fundamentación conjuntista y topológica del espacio celular

Para formalizar el estudio de la computación en sistemas dinámicos discretos, es imperativo establecer la estructura geométrica base sobre la cual operará el sistema. El análisis del comportamiento computacional y de la emergencia de partículas requiere que el medio físico (o su abstracción matemática) esté rigurosamente delimitado bajo estrictos criterios conjuntistas.

2.1.1 Definición del espacio y la unidad celular

Se define el espacio celular como una retícula geométrica infinita y multidimensional, denotada matemáticamente por el conjunto $\mathbb{Z}^d$, donde $\mathbb{Z}$ representa el conjunto de los números enteros y $d \in \mathbb{N}$ determina la dimensionalidad del espacio analizado [5].

Dentro de esta retícula, se define formalmente el concepto de **célula** como la unidad mínima indivisible del modelo de autómata celular. Cada célula posee una posición vectorial discreta e inmutable en el espacio. Sea c una célula cualquiera en el espacio d -dimensional, su ubicación espacial exacta está dada por un vector $\mathbf{v} \in \mathbb{Z}^d$, tal que:

$$\mathbf{v} = (x_1, x_2, \dots, x_d) \quad \text{donde} \quad x_i \in \mathbb{Z} \quad (2.1)$$

Para los propósitos de modelado del presente trabajo, se hará especial énfasis en el análisis del caso bidimensional ($d = 2$). En un espacio bidimensional ortogonal $\mathbb{Z}^2$, la posición de cada célula se simplifica a un par ordenado de coordenadas discretas (i, j) .

2.1.2 Definición axiomática de autómata celular

Un autómata celular (AC) es un sistema dinámico y discreto en el espacio, el tiempo y el estado. Es un modelo que evoluciona mediante pasos de tiempo sincrónicos donde cada elemento evalúa su estado futuro basándose en su entorno local [2]. Matemáticamente, se define como una cuádrupla funcional:

$$AC = (\mathcal{L}, \Sigma, \mathcal{N}, \delta) \quad (2.2)$$

donde:

- $\mathcal{L}$ es el espacio subyacente, constituido por la retícula de dimensión finita (comúnmente $\mathbb{Z}^d$).
- Σ es el alfabeto o conjunto finito de estados elementales.
- $\mathcal{N}$ es la vecindad o subconjunto finito de desplazamientos vectoriales adyacentes a la célula analizada.
- δ es la función de transición local que proyecta el estado actual de la vecindad al estado futuro de la célula central.

2.1.3 Alfabetos finitos y el espacio de estados

La dinámica de la información en un autómata celular se manifiesta a través de la alteración sistemática de los valores asignados a cada coordenada espacial. Se define formalmente un conjunto finito de estados discretos, denominado alfabeto y denotado por el símbolo Σ . Este conjunto contiene íntegramente todos los posibles estados mutuamente excluyentes que una célula $c \in \mathbb{Z}^d$ puede adoptar en un instante de tiempo arbitrario t .

Por definición, el alfabeto cumple con la propiedad de cardinalidad finita $|\Sigma| = k$, para todo $k \in \mathbb{N}$ tal que $k \geq 2$. En su forma más reducida y elemental, particularmente en los dominios de la computación lógica clásica y la electrónica digital, el alfabeto empleado es el binario, donde $\Sigma = \{0, 1\}$.

Adicionalmente a la cardinalidad, se establece la existencia obligatoria de un estado quiescente o estado de reposo absoluto $q_0 \in \Sigma$. El estado quiescente posee la característica de equilibrio local: si una célula arbitraria y la totalidad de sus células adyacentes determinadas por la vecindad $\mathcal{N}$ se encuentran concurrentemente en el

estado q_0 en el tiempo t , la célula central evaluada permanecerá forzosamente en el estado q_0 para el paso temporal siguiente $t + 1$.

2.1.4 Formalización métrica de vecindades acotadas

El flujo y procesamiento de la información en un autómata celular están estrictamente limitados y supeditados a interacciones puramente locales. Para formalizar mecánicamente este principio de localidad, se definen matemáticamente los subconjuntos finitos adyacentes que influyen sobre la función de transición de una célula central.

Una vecindad genérica $\mathcal{N}$ de tamaño n se define como un conjunto finito y ordenado de vectores de desplazamiento relativo desde el origen:

$$\mathcal{N} = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\} \quad \text{donde} \quad \mathbf{v}_i \in \mathbb{Z}^d \quad (2.3)$$

En consecuencia, para una célula central ubicada en la posición absoluta $\mathbf{c} \in \mathbb{Z}^d$, el conjunto coordinado de todas sus células vecinas se obtiene mediante la suma vectorial homóloga:

$$V(\mathbf{c}) = \{\mathbf{c} + \mathbf{v}_i \mid \mathbf{v}_i \in \mathcal{N}\} \quad (2.4)$$

Dentro de la retícula estándar bidimensional $\mathbb{Z}^2$, las configuraciones topológicas más aplicadas se delimitan utilizando métricas de distancia discreta [6]. La vecindad de Moore de radio $r = 1$ comprende a las 8 células adyacentes (delimitadas por la distancia de Chebyshev infinita normada a 1), mientras que la vecindad de von Neumann incluye exclusivamente a las 4 células ortogonales (determinadas por la distancia de Manhattan igual a 1).

2.1.5 El autómata celular hexagonal

Como punto medular de esta investigación, se introduce una variante topológica de profundo interés analítico: el **autómata celular hexagonal**. A diferencia de la retícula cartesiana tradicional $\mathbb{Z}^2$ donde cada célula es modelada como un polígono cuadrangular y comparte límites asimétricos (bordes directos frente a vértices diagonales) con sus 8 vecinos de Moore, la teselación hexagonal plana permite dividir uniformemente el espacio mediante polígonos regulares de seis lados.

Esta propiedad geométrica dota al sistema de una simetría isotrópica inmensamente superior, al garantizar que la distancia proyectada desde el centroide de una célula al centroide de todos y cada uno de sus vecinos inmediatos sea matemáticamente idéntica.

En un autómata celular hexagonal, se define el espacio $\mathcal{L}_{hex}$ como una malla isomórfica a una red reticular triangular bidimensional [7]. La vecindad básica,

equivalente a un radio $r = 1$, se compone estrictamente de las seis células adyacentes que comparten de forma colineal una arista con la célula central.

Formalmente, si se aplica una inmersión de esta red hexagonal sobre un sistema de coordenadas oblicuas bidimensionales (x, y) , la vecindad canónica hexagonal $\mathcal{N}_{hex}$ se modela axiomáticamente mediante el conjunto de los seis vectores de desplazamiento relativo:

$$\mathcal{N}_{hex} = \{(1, 0), (0, 1), (-1, 1), (-1, 0), (0, -1), (1, -1)\} \quad (2.5)$$

El uso del sistema de coordenadas oblicuas permite incrustar la topología de base 6 dentro de los límites matemáticos y computacionales de arreglos matriciales discretos. Esta estructura topológica es fundamental en la disciplina de simulación no convencional, debido a que su uniformidad reduce los sesgos angulares (anisotropía) durante la propagación dinámica de información y modelado de trayectorias de partículas complejas.

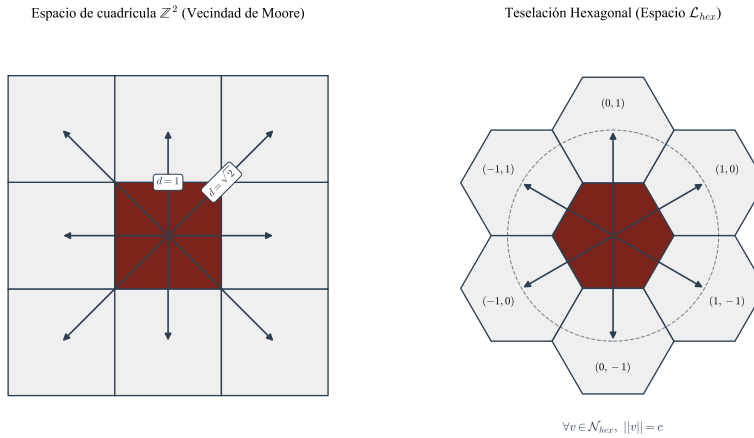


Fig. 2.1 Comparativa geométrica topológica entre el espacio de cuadrícula $\mathbb{Z}^2$ asimétrica y la teselación celular hexagonal isótropa. Se detallan los vectores de desplazamiento $\mathbf{v}_i$ que definen la vecindad adyacente $\mathcal{N}_{hex}$.

2.2 Funciones de transición y evolución espacio-temporal

Una vez establecida la base estructural del espacio celular en la sección anterior, es necesario formalizar el mecanismo que rige el cambio de estado de las células a través del tiempo. La transición de una configuración estática a un sistema dinámico se logra mediante la definición de reglas de evolución locales, las cuales determinan el comportamiento emergente del autómata celular en una escala global.

2.2.1 La función de transición local

La dinámica de un autómata celular está gobernada por una función de transición local δ , la cual actúa como un mapeo determinista entre el estado actual de la vecindad de una célula y su estado futuro. Sea $\sigma_c^t \in \Sigma$ el estado de una célula c en el instante de tiempo t . La evolución de dicha célula se define formalmente como:

$$\sigma_c^{t+1} = \delta(\sigma_{v_1}^t, \sigma_{v_2}^t, \dots, \sigma_{v_n}^t) \quad (2.6)$$

donde $(\sigma_{v_1}^t, \dots, \sigma_{v_n}^t)$ representa la configuración de estados de las células contenidas en la vecindad $\mathcal{N}$ en el tiempo t . Es fundamental subrayar que la función δ es idéntica para todas las células de la retícula y permanece invariante ante traslaciones espaciales y temporales [5]. Esta uniformidad garantiza que las leyes "físicas" del universo digital sean consistentes en todo el dominio $\mathbb{Z}^d$.

2.2.2 Configuraciones globales y el mapeo paralelo

Para describir el estado del sistema en su totalidad, se introduce el concepto de **configuración global** C , la cual se define como una función que asigna un estado del alfabeto a cada célula de la retícula: $C : \mathbb{Z}^d \rightarrow \Sigma$. El conjunto de todas las configuraciones posibles se denomina espacio de configuraciones y se denota por $\Sigma^{\mathbb{Z}^d}$.

La evolución global del autómata celular se formaliza mediante el mapeo de configuración global $G : \Sigma^{\mathbb{Z}^d} \rightarrow \Sigma^{\mathbb{Z}^d}$. Este mapeo aplica simultáneamente la función de transición local δ en cada posición de la retícula, de modo que la transición de una configuración C_t a C_{t+1} ocurre de forma síncrona y paralela:

$$C_{t+1} = G(C_t) \quad (2.7)$$

Esta transición de lo estático (la configuración individual) a lo dinámico (la secuencia de configuraciones) permite observar la propagación de información y la formación de estructuras complejas. En el caso de los autómatas celulares complejos, este mapeo global es el que da lugar a la fenomenología de partículas o *gliders*, los cuales pueden interpretarse como entidades computacionales que se desplazan sobre el espacio de estados [2].

2.2.3 Discretización del tiempo y trayectoria evolutiva

El tiempo en este modelo se considera una variable discreta $t \in \mathbb{N} \cup \{0\}$. La evolución completa del sistema partiendo de una configuración inicial C_0 se describe mediante la órbita o trayectoria:

$$C_0, G(C_0), G(G(C_0)), \dots, G^t(C_0) \quad (2.8)$$

En la práctica computacional, estas evoluciones suelen representarse mediante diagramas espacio-temporales, donde una dimensión representa el espacio y otra el tiempo. Para el caso bidimensional ($d = 2$), la evolución se visualiza como una sucesión de capas o fotogramas (*frames*), permitiendo el análisis topológico de las colisiones entre partículas, las cuales constituyen la base para la implementación de compuertas lógicas en sistemas complejos.

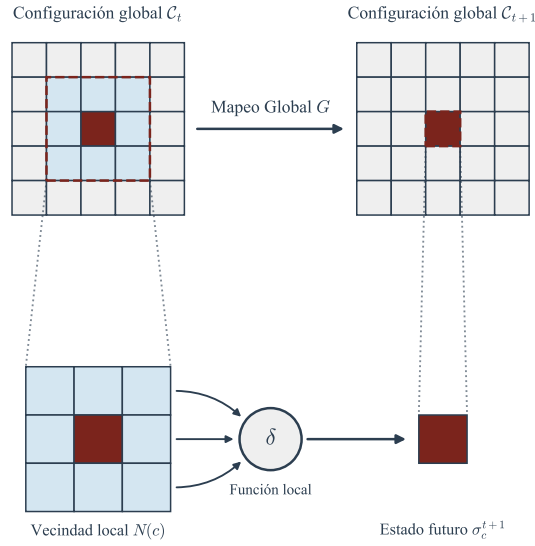


Fig. 2.2 Representación del mapeo de estados locales a la evolución de la configuración global del sistema.

2.3 Dinámica global y clasificación cualitativa del espacio de fases

La iteración continua del mapeo global G sobre una configuración inicial C_0 genera una trayectoria única en el espacio de configuraciones $\Sigma^{\mathbb{Z}^d}$. Para comprender sistemáticamente los patrones de convergencia o divergencia a largo plazo, resulta indispensable estudiar el autómata celular desde la perspectiva de los sistemas dinámicos discretos, analizando la topología de su espacio de fases.

2.3.1 Atractores espaciales y estabilidad asintótica

Se define el espacio de fases discreto de un autómata celular finito como un grafo dirigido, donde cada vértice representa una configuración global posible $C \in \Sigma^{\mathbb{Z}^d}$ y cada arista dirigida indica una transición lícita unívoca dictada por la función G . Dado que para una retícula finita el número total de configuraciones es discreto y acotado a $|\Sigma|^N$ (donde N es el número total de células), el determinismo de la función de transición garantiza que cualquier trayectoria temporal eventualmente cruzará por una configuración previamente visitada, ingresando a un ciclo invariante [3].

Un estado invariante, o punto fijo asintótico, se formaliza como una configuración C_f que permanece inalterada bajo la aplicación del mapeo global:

$$G(C_f) = C_f \quad (2.9)$$

De manera análoga, se define un ciclo límite de periodo p como una secuencia cerrada de p configuraciones distintas, donde el sistema retorna iterativamente a su estado inicial tras p pasos de tiempo discretos:

$$G^p(C_c) = C_c \quad \text{con } p \geq 2 \quad (2.10)$$

El conjunto de configuraciones que conforman el punto fijo o el ciclo límite se denomina **atractor**. Asimismo, la *cuenca de atracción* se define axiomáticamente como el conjunto de todas las configuraciones iniciales cuyas trayectorias transitorias (pre-periodos) convergen asintóticamente hacia dicho atractor. El estudio de estos sumideros topológicos permite determinar la disipación de información en el sistema celular.

2.3.2 Formalización de la clasificación de Wolfram

A partir del análisis fenomenológico del espacio de fases y la evolución espacio-temporal a partir de configuraciones iniciales aleatorias, se establece la taxonomía universal propuesta por Stephen Wolfram [2]. Esta clasificación divide el dominio algorítmico de los autómatas celulares en cuatro clases cualitativas de comportamiento global:

- **Clase I (Convergencia homogénea):** La evolución del sistema converge rápidamente hacia un atractor espacialmente homogéneo, equivalente a un punto fijo trivial (usualmente el estado quiescente absoluto). Cualquier configuración de información inicial es destruida en un tiempo finito.
- **Clase II (Estructuras periódicas):** El sistema evoluciona hacia un conjunto de estructuras espacialmente separadas y periódicas (ciclos límite de periodo corto). La propagación de la información intercelular queda estrictamente acotada a una vecindad confinada.

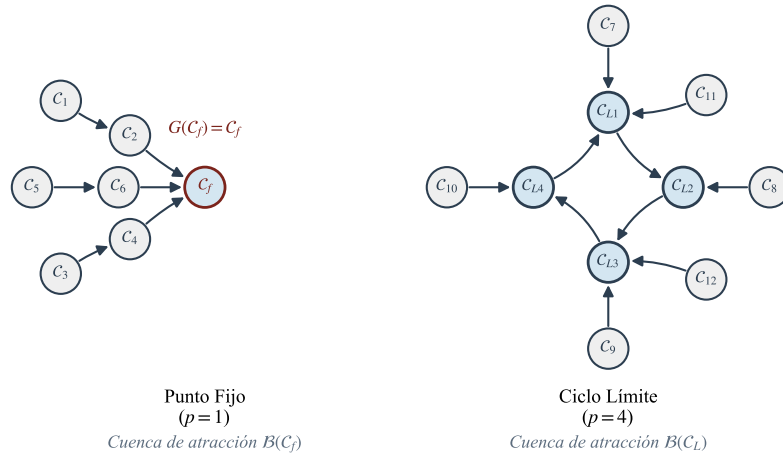


Fig. 2.3 Representación esquemática del espacio de fases discreto. Se observan los estados transitorios que conforman las cuencas de atracción convergiendo en estados estables y ciclos límite.

- **Clase III (Comportamiento caótico):** La trayectoria conduce a patrones pseudoaleatorios y aperiódicos continuos. Las perturbaciones locales se propagan indefinidamente a lo largo del espacio, generando una extrema sensibilidad a las condiciones iniciales, en un fenómeno análogo a la emergencia de atractores extraños.
- **Clase IV (Comportamiento complejo):** El espacio celular exhibe una coexistencia altamente correlacionada de orden y caos, estabilizándose en una frontera de transición crítica. Se caracteriza por la emergencia de estructuras localizadas, persistentes y espacialmente móviles (*gliders*) que interaccionan y colisionan de forma no trivial.

Para los propósitos de la presente investigación, enfocada en la computabilidad y el análisis de dinámicas como *Spiral* y *Beehive*, el interés axiomático recae de forma exclusiva sobre la **Clase IV**. Es únicamente en este régimen analítico donde la información codificada en estructuras móviles no se disipa de inmediato (como en las Clases I y II) ni se degrada en ruido inoperable (Clase III). La preservación de trayectorias en los atractores de la Clase IV asegura colisiones deterministas, proveyendo así la base teórica indispensable para emular lógica booleana y síntesis de operadores de cómputo universal.

2.4 Teoría de la computabilidad sobre sistemas dinámicos discretos

La caracterización de los autómatas celulares como sistemas dinámicos permite no solo el modelado de fenómenos físicos, sino también la construcción de un paradigma de procesamiento de información alternativo a la arquitectura de Von Neumann tradicional. En esta sección, se establece la inmersión de la Teoría de la Computación dentro de los espacios celulares, justificando formalmente su capacidad para realizar cálculos universales a través de la interacción de estados discretos.

2.4.1 Equivalencia algorítmica y Máquinas de Turing

La justificación de los autómatas celulares como modelos de cómputo reside en su capacidad para emular el comportamiento de una Máquina de Turing (MT). Se define formalmente que un autómata celular es Turing-completo si existe un mapeo que permita representar la cinta, el alfabeto y la función de transición de una MT dentro de la configuración global del autómata [5].

A diferencia del modelo secuencial de la MT, donde un cabezal altera una celda a la vez, el espacio celular $\mathbb{Z}^d$ opera bajo un esquema de procesamiento masivamente paralelo. No obstante, Von Neumann demostró axiomáticamente que es posible construir un autómata celular con un número finito de estados (específicamente 29 estados en su modelo original) capaz de soportar tanto la computación universal como la autorreplicación [1]. En este sentido, la "computación" en un espacio discreto se redefine como la evolución de una configuración inicial C_0 (datos de entrada) hacia una configuración final C_t (resultado) tras la aplicación iterativa del mapeo global G .

2.4.2 Emergencia de operadores lógicos y universalidad computacional

Para que un sistema dinámico discreto sea considerado un soporte de computación universal, debe poseer la capacidad intrínseca de sintetizar funciones booleanas fundamentales. En el marco de los autómatas celulares complejos (Clase IV de Wolfram), esta capacidad no se implementa mediante componentes electrónicos fijos, sino a través de la dinámica de colisiones entre partículas emergentes.

Se axiomatiza que la capacidad de procesamiento en estos sistemas se sustenta en tres pilares lógicos:

1. **Almacenamiento:** Representado por estructuras estáticas o periódicas que preservan un bit de información en una coordenada espacial fija.
2. **Transmisión:** Ejecutada por los *gliders* o estructuras móviles que trasladan información a través de la retícula $\mathbb{Z}^2$.

3. **Procesamiento:** Manifestado en los puntos de colisión, donde la interacción de dos o más partículas produce una resultante determinista que emula el comportamiento de una compuerta lógica (e.g., AND, OR, NOT) [4].

Es posible observar que la complejidad de las reglas *Spiral* y *Beehive*, objeto de esta investigación, permite la existencia de estos tres pilares. La construcción de un sistema de cómputo universal sobre estas reglas implica, por tanto, el diseño de una "computación basada en colisiones", donde la trayectoria de los gliders es el equivalente geométrico a las líneas de datos en un circuito integrado convencional [6].

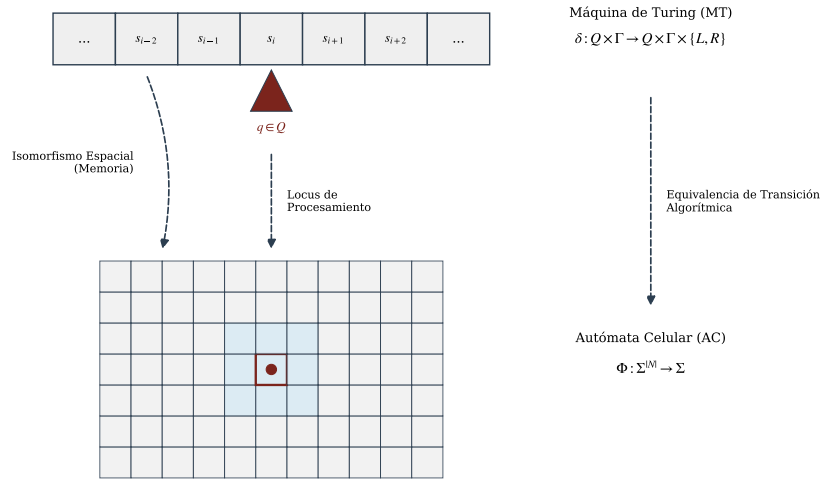


Fig. 2.4 Isomorfismo entre el modelo de Máquina de Turing y la evolución dinámica de un autómata celular bidimensional.

2.5 Reversibilidad topológica y dinámica de la información

La evolución de un autómata celular no solo describe la transformación geométrica de patrones en el espacio discreto, sino que también dicta las leyes de conservación de la información dentro del sistema. La capacidad de una topología para realizar cálculos lógicos está íntimamente ligada a la preservación o disipación de los datos iniciales. Por consiguiente, es imperativo analizar la naturaleza del mapeo global desde una perspectiva de invertibilidad funcional.

2.5.1 Inyectividad, suprayectividad y teoremas fundamentales

Se establece que un autómata celular es **globalmente reversible** si y solo si su función de transición global $G : \Sigma^{\mathbb{Z}^d} \rightarrow \Sigma^{\mathbb{Z}^d}$ es biyectiva. En términos fenomenológicos, la reversibilidad topológica implica un determinismo bidireccional estricto: así como el estado futuro se calcula inequívocamente a partir del presente, la configuración pasada C_{t-1} puede ser deducida de manera unívoca a partir de la configuración actual C_t [4].

Para que la función G sea biyectiva, debe cumplir simultáneamente con dos propiedades matemáticas estructurales:

1. **Inyectividad:** No existen dos configuraciones iniciales distintas que evolucionen hacia la misma configuración resultante. Formalmente, si $C_A \neq C_B$, entonces $G(C_A) \neq G(C_B)$. La inyectividad garantiza que no hay pérdida ni colapso de información durante la transición de estados.
2. **Suprayectividad:** Toda configuración concebible dentro del espacio de fases $\Sigma^{\mathbb{Z}^d}$ posee al menos una configuración predecesora válida. Es decir, para todo estado $C \in \Sigma^{\mathbb{Z}^d}$, existe un C' tal que $G(C') = C$.

Dentro de la teoría formal de espacios discretos finitos e infinitos, la relación entre ambas propiedades queda fundamentada por el Teorema de Richardson, el cual demuestra axiomáticamente que, para autómatas celulares, la inyectividad del mapeo global G es una condición suficiente para garantizar su suprayectividad, y por ende, su reversibilidad [5].

2.5.2 Entropía discreta y estados Jardín del Edén

Cuando un autómata celular no es suprayectivo, la dinámica del sistema experimenta un fenómeno de irreversibilidad caracterizado por la convergencia de múltiples trayectorias hacia un mismo atractor, colapsando la entropía discreta del sistema. La consecuencia matemática directa de la falta de suprayectividad es la existencia de configuraciones inalcanzables.

A estas configuraciones sin preimagen se les denomina axiomáticamente **estados Jardín del Edén**. Una configuración C_{GE} es un estado Jardín del Edén si no existe ninguna configuración en el dominio espacial que la genere mediante un paso temporal; formalmente:

$$\nexists C \in \Sigma^{\mathbb{Z}^d} \quad \text{tal que} \quad G(C) = C_{GE} \quad (2.11)$$

El Teorema de Moore-Myhill establece una equivalencia lógica profunda: un autómata celular posee estados Jardín del Edén si y solo si su función global G no es inyectiva [3]. Esto significa que la existencia de configuraciones inalcanzables es un síntoma irrefutable de que el sistema destruye información en su evolución temporal (múltiples pasados convergen en un solo presente).

Para los propósitos sintéticos de este Trabajo Terminal, orientados a la emulación computacional mediante colisiones en las reglas complejas *Spiral* y *Beehive*, la irreversibilidad juega un rol paradójicamente constructivo. Si bien los sistemas reversibles conservan la información (asimilables a compuertas lógicas cuánticas o compuertas de Fredkin), el diseño de operaciones booleanas asimétricas tradicionales, como la compuerta lógicas AND u OR discreta, requiere axiomáticamente la disipación de estados (e.g., las entradas $\{0, 1\}$, $\{1, 0\}$ y $\{0, 0\}$ convergen todas a una salida 0 en una compuerta AND). Por consiguiente, la aniquilación mutua de partículas (*gliders*) en los modelos estudiados se modela teóricamente como transiciones irreversibles que generan implícitamente configuraciones Jardín del Edén, dotando al sistema de la capacidad de operar como un sumidero o filtro de información lógico.

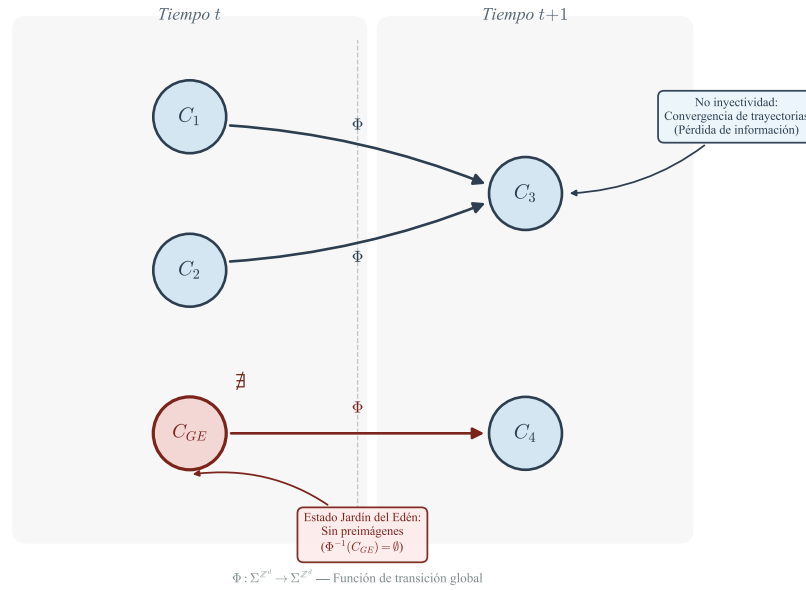


Fig. 2.5 Representación topológica de la irreversibilidad en el espacio de estados. La convergencia de trayectorias ilustra la pérdida de información, mientras que los nodos sin preimágenes denotan estados Jardín del Edén.

Parte II
Morfología, geometría estructural y
equivalencia topológica

Esta segunda parte del documento establece la transición formal entre la teoría abstracta de los autómatas celulares, desarrollada en los Capítulos 1 y 2, y su representación geométrica computable. El objetivo de esta sección es definir los modelos matemáticos y topológicos necesarios para proyectar la dinámica de una retícula hexagonal sobre la arquitectura ortogonal de la memoria de un equipo de cómputo convencional, garantizando en todo momento la rigurosidad analítica y la invariancia espacial.

En el Capítulo 3, se formalizan las estructuras algebraicas que rigen el espacio discreto bidimensional. Se modela la retícula como un grupo abeliano bajo la adición y se definen las métricas analíticas fundamentales para la tesela hexagonal. De igual forma, se establece el modelo algebraico inverso para transformar el espacio continuo $\mathbb{R}^2$ a coordenadas discretas. Esto permite demostrar teóricamente que el algoritmo de actualización topológica opera en un tiempo asintóticamente lineal $O(n)$ y con requerimientos espaciales de $O(n)$, justificando empírica y formalmente el desarrollo del simulador en C/C++.

Posteriormente, en el Capítulo 4, se aborda el problema central de la representación en memoria: la transformación geométrica entre topologías. Se definen las ecuaciones de mapeo afín para proyectar la vecindad hexagonal pura hacia un tensor ortogonal de subred 2×2 (denotado como $\mathcal{T}_B$). A través de una demostración deductiva, se comprueba que la inactividad intencional de dos aristas en este bloque ortogonal previene la pérdida de simetría en la propagación de la información.

En conjunto, el modelado algebraico y las demostraciones topológicas de esta parte constituyen el núcleo del motor matemático del proyecto. Este bloque garantiza estructuralmente que ninguna regla o propiedad de computabilidad se degrade al ser simulada, estableciendo el sustrato algorítmico indispensable para el análisis cinemático de partículas (*gliders*) y el cálculo de interacciones locales booleanas que se detallarán en el Capítulo 5.

Capítulo 3

Geometría computacional y formación algebraica para la representación espacial

Resumen En este capítulo se establece la transición entre la abstracción lógica de los autómatas celulares y su representación geométrica tangible. Se formalizan las estructuras algebraicas que sustentan las retículas bidimensionales, definiendo el espacio discreto como un grupo abeliano bajo la adición. Asimismo, se analizan las métricas de distancia necesarias para delimitar vecindades y se desarrolla la geometría analítica de la tesela hexagonal. Este marco teórico constituye la base fundamental para el desarrollo del simulador en C/C++, permitiendo el mapeo riguroso de coordenadas discretas a espacios visuales continuos.

3.1 Estructuras algebraicas para retículas espaciales

La comprensión de un autómata celular (AC) como sistema dinámico discreto requiere, en primera instancia, de una definición unívoca del espacio donde ocurre la evolución. Si bien en capítulos anteriores se abordó la dinámica de estados, en esta sección se formaliza el soporte topológico sobre el cual se asientan dichas celdas.

3.1.1 Definición axiomática del espacio bidimensional discreto

Se define la retícula celular, denotada por $\mathcal{L}$, como el conjunto de puntos en un espacio euclidiano que poseen coordenadas enteras. Para los fines de este Trabajo Terminal, se restringe el estudio al plano discreto bidimensional, el cual se identifica con el producto cartesiano de los enteros consigo mismo:

$$\mathcal{L} = \mathbb{Z}^2 = \{(i, j) \mid i, j \in \mathbb{Z}\} \quad (3.1)$$

Cada par ordenado $c = (i, j) \in \mathbb{Z}^2$ representa la posición de una celda individual dentro del sistema. Es fundamental observar que este conjunto posee una estructura de *grupo abeliano* bajo la operación de adición vectorial estándar.

Definition 3.1 Sea $c_1 = (i_1, j_1)$ y $c_2 = (i_2, j_2)$ dos elementos de $\mathbb{Z}^2$. La operación suma $+$: $\mathbb{Z}^2 \times \mathbb{Z}^2 \rightarrow \mathbb{Z}^2$ se define como:

$$c_1 + c_2 = (i_1 + i_2, j_1 + j_2) \quad (3.2)$$

Esta estructura algebraica garantiza el cumplimiento de las siguientes propiedades fundamentales, las cuales son esenciales para la invarianza por traslación de las reglas de transición [5]:

1. **Clausura:** Para todo $c_1, c_2 \in \mathbb{Z}^2$, el resultado de $c_1 + c_2$ es un elemento que pertenece a $\mathbb{Z}^2$.
2. **Asociatividad:** La agrupación de términos no altera el resultado de la suma de múltiples posiciones.
3. **Existencia del elemento neutro:** Existe un vector origen $\mathbf{0} = (0, 0) \in \mathbb{Z}^2$ tal que para toda celda c , $c + \mathbf{0} = c$.
4. **Existencia de elementos inversos:** Para cada celda $c = (i, j)$, existe un inverso $-c = (-i, -j)$ tal que $c + (-c) = \mathbf{0}$.
5. **Conmutatividad:** El orden de la suma no altera la posición resultante, lo que simplifica el cálculo de desplazamientos relativos en la retícula.

La naturaleza de grupo abeliano de $(\mathbb{Z}^2, +)$ permite que cualquier vecindad sea definida de manera relativa a una celda central mediante un conjunto fijo de vectores de desplazamiento, asegurando que la topología del espacio sea uniforme en todos sus puntos.

3.1.2 Topología métrica de la red celular

Para delimitar el alcance de la interacción entre celdas (vecindad), es imperativo introducir una función de distancia d que actúe sobre la retícula. Una métrica en $\mathbb{Z}^2$ es una función $d : \mathbb{Z}^2 \times \mathbb{Z}^2 \rightarrow \mathbb{R}_{\geq 0}$ que satisface los axiomas de identidad de indiscernibles, simetría y desigualdad triangular.

En el contexto de los autómatas celulares, la elección de la métrica define la geometría de la vecindad. Se consideran tres métricas fundamentales inducidas por la norma L_p :

1. **Distancia de Manhattan (L_1):** Define la distancia como la suma de las diferencias absolutas de sus coordenadas. Es la métrica natural para vecindades tipo Von Neumann.

$$d_1(c_1, c_2) = |i_1 - i_2| + |j_1 - j_2| \quad (3.3)$$

2. **Distancia de Chebyshev (L_∞):** Representa el máximo de las diferencias absolutas. Esta métrica es la base para la construcción de vecindades de Moore [2].

$$d_\infty(c_1, c_2) = \max(|i_1 - i_2|, |j_1 - j_2|) \quad (3.4)$$

3. **Distancia Euclidiana (L_2):** Proporciona la distancia física en línea recta en el plano continuo $\mathbb{R}^2$.

$$d_2(c_1, c_2) = \sqrt{(i_1 - i_2)^2 + (j_1 - j_2)^2} \quad (3.5)$$

La relevancia de estas métricas radica en la definición formal de la vecindad de radio r para una celda c , denotada como $\mathcal{N}_r(c)$, la cual se establece como el conjunto de celdas cuya distancia a c es menor o igual a r :

$$\mathcal{N}_r(c) = \{c' \in \mathbb{Z}^2 \mid d(c, c') \leq r\} \quad (3.6)$$

Es posible observar que, para una retícula cuadrada convencional, una vecindad de Von Neumann corresponde a una bola cerrada bajo la métrica L_1 , mientras que una vecindad de Moore corresponde a una bola cerrada bajo la métrica L_∞ . En el caso de la topología hexagonal que motiva este estudio, se requerirá una generalización de estas distancias para adaptarse a la simetría de seis ejes, lo cual se desarrollará analíticamente en la Sección 3.2.

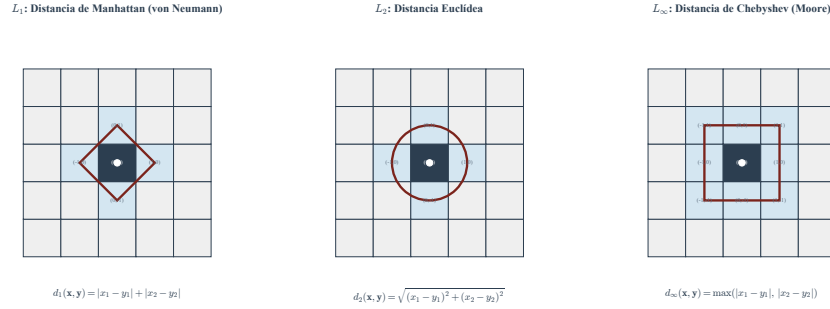


Fig. 3.1 Representación geométrica de las vecindades inducidas por distintas métricas en $\mathbb{Z}^2$. Se observa cómo la elección de la distancia determina la morfología de la interacción local.

3.2 Geometría analítica de la tesela fundamental hexagonal

Tras la formalización de la retícula como un grupo abeliano, resulta imperativo caracterizar la geometría de la unidad mínima de información: la celda hexagonal. A diferencia de las teselas cuadrangulares, el hexágono regular ofrece una simetría

superior y una equidistancia entre vecinos que favorece la emergencia de estructuras dinámicas complejas [7].

3.2.1 Parametrización geométrica de la celda unitaria en $\mathbb{R}^2$

Sea R el *circunradio* de un hexágono regular, definido como la distancia desde el centro geométrico hacia cualquiera de sus vértices. Para propósitos de este modelado, se considera una orientación de “punta superior” (*pointy-top*), donde el hexágono se encuentra centrado en el origen coordenado $(0, 0)$.

Los seis vértices que delimitan la frontera de la celda, denotados por el conjunto $\mathcal{V} = \{V_0, V_1, \dots, V_5\}$, pueden ser calculados mediante identidades trigonométricas vectoriales. Dado que el hexágono regular divide el plano en ángulos de $\frac{\pi}{3}$ radianes (60°), la posición de cada vértice V_k está determinada por:

$$V_k = \begin{pmatrix} x_k \\ y_k \end{pmatrix} = \begin{pmatrix} R \cos \left(\frac{\pi}{6} + \frac{k\pi}{3} \right) \\ R \sin \left(\frac{\pi}{6} + \frac{k\pi}{3} \right) \end{pmatrix}, \quad \text{para } k = 0, 1, \dots, 5. \quad (3.7)$$

Es posible derivar la relación entre el circunradio R y la *apotema* (denotada como a), la cual representa la distancia mínima desde el centro hacia el punto medio de cualquiera de sus lados. Mediante el análisis del triángulo rectángulo formado por el radio, la apotema y la mitad de un lado, se establece que:

$$a = R \cos \left(\frac{\pi}{6} \right) = \frac{\sqrt{3}}{2} R \quad (3.8)$$

Esta constante geométrica es fundamental para el algoritmo de renderizado, pues define el factor de escala en el eje de las abscisas al realizar el empaquetamiento de la red, vease Fig. 3.2.

3.2.2 Formulación del dominio poligonal y ecuaciones de contorno

Para garantizar una detección rigurosa de pertenencia de puntos —proceso crítico en la interfaz gráfica del simulador—, el interior de la celda se modela como un conjunto convexo $\mathcal{H} \subset \mathbb{R}^2$. Este dominio se define matemáticamente como la intersección de seis semiespacios cerrados.

Sea $P = (x, y)$ un punto arbitrario en el plano. Dicho punto pertenece al dominio de la celda hexagonal si y solo si satisface el siguiente sistema de inecuaciones lineales simultáneas, derivadas de la simetría axial del polígono:

$$\mathcal{H} = \left\{ (x, y) \in \mathbb{R}^2 \mid \begin{cases} |y| \leq \frac{\sqrt{3}}{2} R \\ \frac{\sqrt{3}}{2} |x| + \frac{1}{2} |y| \leq \frac{\sqrt{3}}{2} R \end{cases} \right\} \quad (3.9)$$

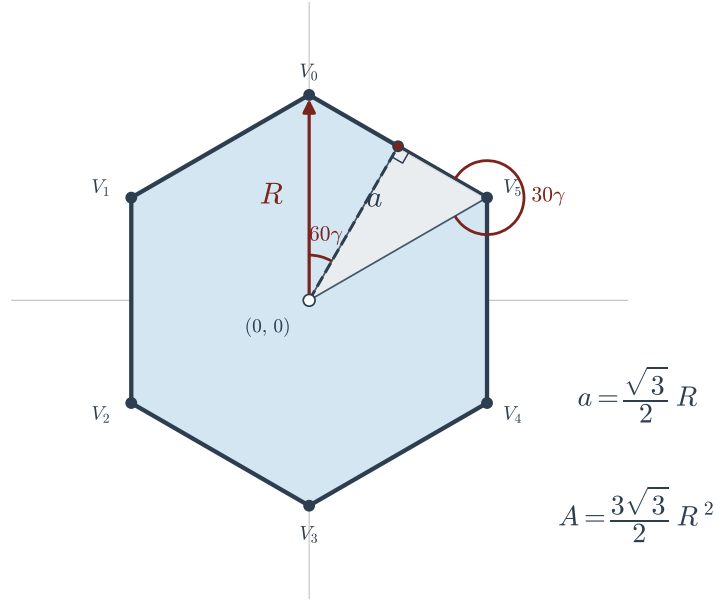


Fig. 3.2 Geometría analítica de la tesela hexagonal unitaria. Se destacan las relaciones trigonométricas entre el circunradio y la apotema.

La expresión 3.9 permite una verificación computacionalmente eficiente. La primera inecuación delimita la extensión vertical de la celda respecto a sus bordes horizontales, mientras que la segunda inecuación, de naturaleza escalar, restringe el espacio mediante las pendientes de las aristas laterales.

Este modelo algebraico constituye la base analítica para el *mapeo inverso* que se discutirá en secciones posteriores: dada una coordenada de pantalla producida por un evento de ratón, el sistema debe resolver este conjunto de inecuaciones para determinar la celda (i, j) correspondiente en el espacio de estados.

Resulta de interés observar que este planteamiento no depende de una búsqueda iterativa, sino de una evaluación directa de la norma del vector de posición, lo cual optimiza la complejidad temporal del simulador a tiempo constante $O(1)$ por cada consulta de pertenencia.

3.3 Transformaciones afines y mapeo de coordenadas globales

Una vez caracterizada la topología de la celda unitaria hexagonal, resulta imperativo establecer los mecanismos formales para proyectar una pluralidad de estas teselas sobre el plano euclidiano. La transición de la abstracción algebraica de la retícula $\mathbb{Z}^2$ hacia una representación gráfica continua en $\mathbb{R}^2$ requiere la definición de operadores

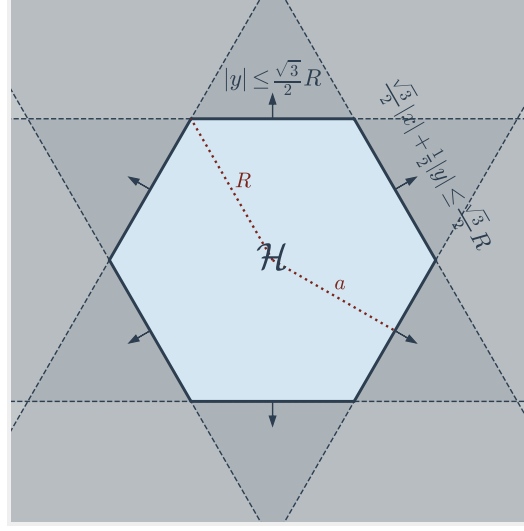


Fig. 3.3 Representación del dominio $\mathcal{H}$ como intersección de semiplanos. Las regiones sombreadas indican el cumplimiento de las restricciones lineales.

matriciales que garanticen una biyección espacial sin superposiciones ni vacíos, logrando un teselado regular perfecto.

3.3.1 Operadores matriciales para la traslación espacial

Sea un par ordenado $(i, j) \in \mathbb{Z}^2$ que denota el índice discreto de una celda en el espacio de estados. Se define la transformación afín $\mathcal{T} : \mathbb{Z}^2 \rightarrow \mathbb{R}^2$ como la función que mapea las coordenadas lógicas hacia el centro geométrico continuo (x, y) del hexágono correspondiente en el plano cartesiano.

Para una retícula hexagonal orientada en configuración de *punta superior* (*pointy-top*), los vectores base del espacio no son ortogonales. Un desplazamiento unitario en el eje discreto de las abscisas (i) corresponde a un movimiento completamente horizontal, mientras que un desplazamiento unitario en el eje de las ordenadas (j) implica un desplazamiento oblicuo.

Esta proyección se formaliza algebraicamente mediante la multiplicación de un vector de coordenadas lógicas por una matriz de transformación M , escalada por el circunradio R :

$$\begin{pmatrix} x \\ y \end{pmatrix} = R \begin{pmatrix} \sqrt{3} & \frac{\sqrt{3}}{2} \\ 0 & \frac{3}{2} \end{pmatrix} \begin{pmatrix} i \\ j \end{pmatrix} \quad (3.10)$$

Es posible observar que el determinante de la matriz de transformación es $\Delta = \frac{3\sqrt{3}}{2}$, un valor distinto de cero que garantiza la independencia lineal de los vectores base. Este modelo, conocido formalmente como *sistema de coordenadas axiales*, es matemáticamente elegante debido a que preserva la estructura de grupo abeliano bajo la suma vectorial estándar. Sin embargo, su mapeo directo hacia arreglos de memoria bidimensionales en lenguajes como C/C++ genera un desperdicio de espacio debido al sesgo romboidal de la proyección [7].

3.3.2 Sistemas paramétricos de coordenadas compensadas (*Offset Coordinates*)

Para resolver la disparidad entre la topología de las estructuras de datos ortogonales en la memoria computacional y la geometría no ortogonal de la retícula hexagonal, se introduce el sistema de *coordenadas compensadas*. Este enfoque modela geoméricamente el empaquetamiento óptimo de la malla aplicando una traslación condicionada por la paridad del índice de fila o columna.

Dado que la arquitectura topológica del simulador se orienta por filas horizontales, se define analíticamente el sistema compensado de fila impar (conocido en la literatura como sistema *odd-r*). En este modelo, las celdas de las filas con índice j impar sufren un desplazamiento horizontal equivalente a la apotema $a = \frac{\sqrt{3}}{2}R$.

Sea $P(j) = j \pmod{2}$ la función indicadora de paridad booleana. El mapeo suprayectivo que determina las coordenadas euclidianas del centro del hexágono (x, y) a partir de sus índices (i, j) se define mediante el siguiente sistema de ecuaciones paramétricas:

$$x(i, j) = \left(i + \frac{P(j)}{2} \right) \sqrt{3}R \quad (3.11)$$

$$y(i, j) = j \left(\frac{3}{2}R \right) \quad (3.12)$$

Este sistema de ecuaciones revela las propiedades de empaquetamiento de la red. La distancia horizontal entre centros adyacentes en la misma fila es estrictamente $2a = \sqrt{3}R$, mientras que la distancia vertical entre filas consecutivas es $\frac{3}{2}R$, lo que provoca un anidamiento (*interlocking*) de las celdas. Geométricamente, esta disposición es isomorfa al empaquetamiento denso de círculos en el plano bidimensional, asegurando que cada celda equidiste perfectamente de sus seis vecinos inmediatos [2].

Cabe destacar que, mediante rotaciones afines, es posible derivar el modelo complementario de columnas compensadas (*even-q* u *odd-q*) para hexágonos con orientación de borde plano (*flat-top*), aunque para los fines de este Trabajo Terminal, la axiomatización se centra en la simetría horizontal dictada por el sistema *odd-r*.

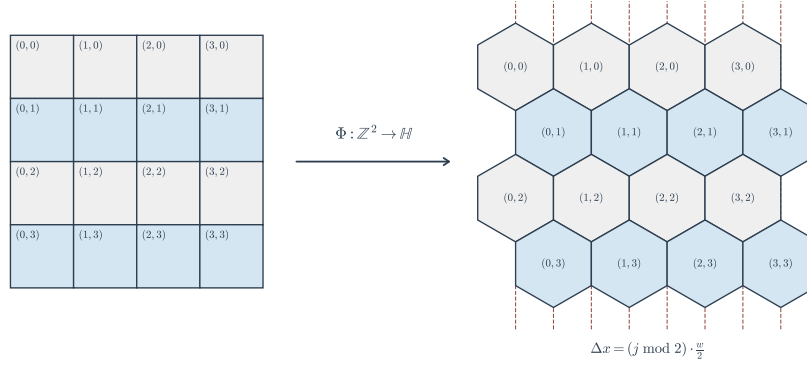


Fig. 3.4 Modelado geométrico del sistema de coordenadas compensadas *odd-r*. Se ilustra cómo la paridad de la fila j determina la traslación sobre el eje de las abscisas, emulando un empaquetamiento óptimo continuo a partir de índices discretos.

3.4 Topologías compactas y axiomatización de fronteras

La formulación teórica de un autómata celular asume, por definición, un espacio de estados infinito $\mathbb{Z}^2$. Sin embargo, cualquier emulación computacional está intrínsecamente restringida por una memoria finita. La delimitación abrupta de la retícula genera un conjunto de celdas fronterizas con vecindades incompletas, lo cual introduce perturbaciones (artefactos topológicos) que destruyen las estructuras dinámicas emergentes. Para preservar la simetría espacial y la computabilidad del modelo, resulta imperativo establecer una topología cerrada que elimine conceptualmente los límites del plano [4].

3.4.1 Construcción algebraica del espacio cociente toroidal

Sea una retícula finita de dimensiones $M \times N$, denotada por $\mathcal{L}_{M,N}$, donde los índices discretos operan en los rangos $i \in \{0, 1, \dots, M-1\}$ y $j \in \{0, 1, \dots, N-1\}$. Para estructurar la transición matemática de esta malla plana acotada hacia una variedad continua, se recurre a la teoría de clases de equivalencia mediante aritmética modular.

Se define una relación de equivalencia $\sim$ sobre el espacio infinito $\mathbb{Z}^2$ tal que dos coordenadas arbitrarias (i, j) y (i', j') son topológicamente indistinguibles si y solo si sus componentes son congruentes módulo las dimensiones de la retícula:

$$(i, j) \sim (i', j') \iff i \equiv i' \pmod{M} \quad \wedge \quad j \equiv j' \pmod{N} \quad (3.13)$$

Bajo esta relación, la clausura topológica de la matriz de estados finita se define como el espacio cociente:

$$\mathbb{T}^2 \cong \mathbb{Z}^2 / (M\mathbb{Z} \times N\mathbb{Z}) \quad (3.14)$$

Este espacio cociente $\mathbb{T}^2$ es isomorfo a un toroide bidimensional discreto. Geométricamente, esta operación equivale a identificar el borde derecho de la matriz con el borde izquierdo (formando un cilindro), y posteriormente unir el borde superior con el inferior. La consecuencia axiomática fundamental de este mapeo es que toda celda en $\mathbb{T}^2$, sin excepción, posee una vecindad completa y simétrica. El espacio carece de fronteras, garantizando la invarianza por traslación de la función de transición global en la totalidad del dominio simulado.

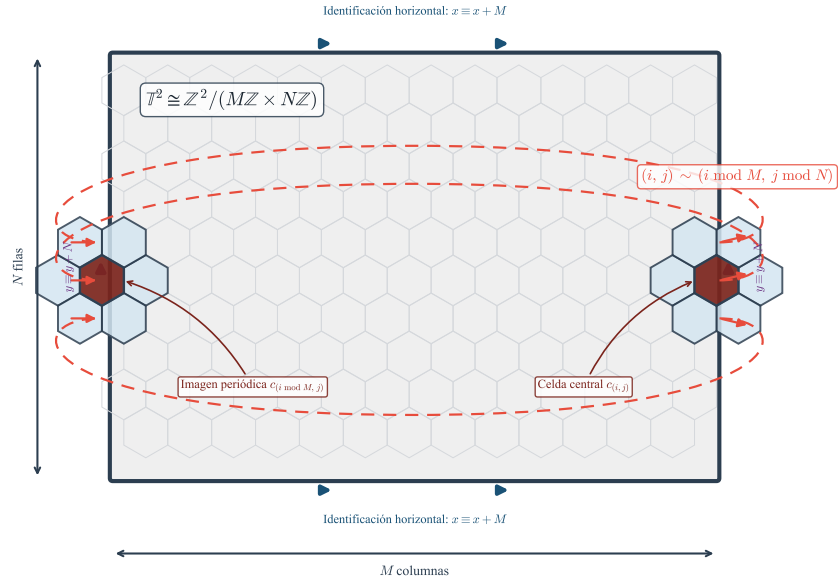


Fig. 3.5 Representación del espacio cociente $\mathbb{T}^2$. Las fronteras opuestas se identifican algebraicamente para construir una topología continua libre de condiciones de borde.

3.4.2 Ecuaciones de transición para condiciones de contorno periódicas

La viabilidad de un autómata celular complejo como sustrato de computación universal depende críticamente de la propagación ininterrumpida de información a través del espacio [2]. Esta información viaja encapsulada en estructuras móviles (frecuentemente denominadas *gliders*). Es indispensable demostrar que el mapeo toroidal conserva la cinemática de dichas partículas.

Sea $G(t) \subset \mathbb{T}^2$ el conjunto de celdas activas que conforman un glider en el instante t . Un glider se define rigurosamente por su periodo $p \in \mathbb{N}^+$ y su vector de traslación espacial $\mathbf{v} = (\Delta i, \Delta j)$, de tal forma que su evolución en un espacio infinito $\mathbb{Z}^2$ satisface $G(t + p) = G(t) + \mathbf{v}$ [7].

Al confinar esta dinámica en el espacio cociente, la función de transición local δ evalúa las vecindades utilizando distancias geodésicas sobre el toroide. La posición de la estructura tras p evoluciones, sujeta a las condiciones de contorno periódicas, está dictada por el mapeo biyectivo:

$$G(t + p) = \{((i + \Delta i) \bmod M, (j + \Delta j) \bmod N) \mid (i, j) \in G(t)\} \quad (3.15)$$

Dado que la operación módulo es distributiva respecto a la adición y preserva la distancia local $d(c_1, c_2)$ entre cualquier par de celdas vecinas a través de las "costuras" del toroide, la morfología de G no experimenta distorsión topológica alguna al cruzar los límites M o N .

Matemáticamente, esto demuestra que el vector de traslación $\mathbf{v}$ se conserva de manera inercial. A largo plazo, cualquier glider que no colisione trazará una trayectoria cerrada cíclica (una curva geodésica discreta) sobre la variedad $\mathbb{T}^2$, retornando eventualmente a su configuración y posición inicial. Esta propiedad de *ciclos límite* no solo valida la estabilidad del modelo computacional, sino que garantiza que las colisiones lógicas programadas entre múltiples gliders operen con determinismo estricto, independientemente de su localización global en el espacio simulado.

3.5 Complejidad computacional del mapeo topológico y renderizado

La formalización algebraica y geométrica desarrollada en las secciones previas carecería de utilidad práctica si no pudiera ser proyectada y manipulada eficientemente en un entorno computacional. El desarrollo de un simulador visual en C/C++ requiere, por una parte, la capacidad de identificar estados a partir de la interacción continua del usuario (mapeo inverso) y, por otra, asegurar que el procesamiento paralelo inherente de los autómatas celulares se ejecute bajo cotas asintóticas viables.

3.5.1 Mapeo espacial inverso y localización de puntos

Durante la ejecución del simulador, la interfaz gráfica permite la manipulación del espacio de estados mediante eventos de puntero. Esto plantea un problema de *picking* geométrico: dado un punto continuo arbitrario $P = (x, y) \in \mathbb{R}^2$, es necesario determinar las coordenadas discretas $(i, j) \in \mathbb{Z}^2$ de la celda hexagonal que lo contiene.

A diferencia de una retícula ortogonal donde la localización se resuelve mediante un truncamiento escalar directo, la teselación no ortogonal requiere la inversión del modelo afín. Partiendo del sistema de coordenadas axiales definido en la Sección 3.3, se establece la matriz de transformación continua-discreta mediante el cálculo de su inversa escalar:

$$\begin{pmatrix} q_f \\ r_f \end{pmatrix} = \frac{1}{R} \begin{pmatrix} \frac{\sqrt{3}}{3} & -\frac{1}{3} \\ 0 & \frac{2}{3} \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \quad (3.16)$$

El resultado de esta operación proyecta el par bidimensional (x, y) en un vector de coordenadas axiales fraccionarias $(q_f, r_f) \in \mathbb{R}^2$. Para obtener el índice discreto unívoco, se proyecta transitoriamente este punto bidimensional hacia un espacio cúbico tridimensional introduciendo una componente auxiliar $s_f = -q_f - r_f$.

Al redondear matemáticamente (q_f, r_f, s_f) al entero más cercano y ajustar la componente con mayor error de truncamiento para garantizar que $q + r + s = 0$, se obtiene la coordenada axial discreta. Finalmente, la restitución de esta terna hacia el sistema de memoria indexado por filas compensadas (*odd-r*), denotado por (i, j) , se formaliza como:

$$j = r, \quad i = q + \left\lfloor \frac{r - (r \bmod 2)}{2} \right\rfloor \quad (3.17)$$

Este modelo algebraico inverso elimina por completo la necesidad de emplear algoritmos de búsqueda iterativa o iteraciones exhaustivas sobre el espacio de estados. La localización geométrica de cualquier perturbación externa se procesa en tiempo constante, sirviendo de fundamento matemático robusto para el motor gráfico en C/C++ [7].

3.5.2 Análisis espacial y temporal de la actualización topológica

Para garantizar la viabilidad computacional de la simulación de colisiones y fenómenos complejos, es necesario someter los algoritmos de actualización global a un análisis de complejidad riguroso. Como se establece en la teoría de análisis de algoritmos introducida por Cormen *et al.* [8], la notación asintótica proporciona una cota paramétrica sobre los recursos consumidos en relación al tamaño de la entrada.

Sea $n = M \times N$ la cardinalidad total de la retícula celular, representando el número de celdas en el espacio cerrado $\mathbb{T}^2$. La función de transición global requiere evaluar

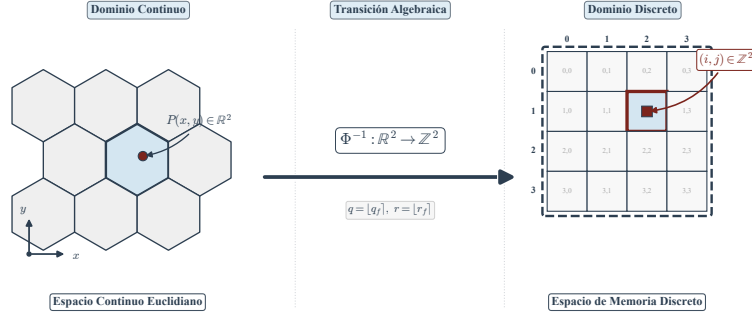


Fig. 3.6 Esquematzación del mapeo algebraico inverso. La transición del dominio continuo de pantalla hacia el arreglo de memoria discreto.

el vecindario de cada elemento. Dado que en una topología hexagonal el tamaño de la vecindad está estrictamente acotado a 6 vecinos colindantes (es decir, una constante independiente de n), el tiempo necesario para resolver las reglas lógicas locales es $O(1)$. En consecuencia, la complejidad en tiempo para actualizar el estado del universo completo en una generación de simulación está dada por:

$$T(n) = \sum_{k=1}^n O(1) = O(n) \quad (3.18)$$

La cota superior lineal $O(n)$ demuestra teóricamente que el algoritmo es asintóticamente óptimo para una arquitectura de procesamiento secuencial unihilo.

Por otro lado, la semántica del autómata celular demanda un sincronismo perfecto; el estado futuro de la red $t + 1$ debe evaluarse pura y exclusivamente sobre la configuración en el instante t . La actualización *in-place* del arreglo de memoria violaría este principio, propagando perturbaciones espúreas. Por lo tanto, el sistema implementado en C/C++ requiere instanciar paralelamente un búfer secundario (técnica conocida como *double buffering*). Esto establece que la complejidad espacial requerida por el motor matemático es:

$$S(n) = O(n) \quad (3.19)$$

La inversión en requerimientos de memoria del modelo secuencial garantiza empíricamente la invariancia por traslación de la red y previene las condiciones de carrera lógicas, siendo el sustento informático indispensable para que la geometría analizada se traduzca correctamente en fenomenología computable.

Capítulo 4

Isomorfismos y transformaciones topológicas entre espacios celulares

Resumen En este capítulo se establece la base formal para la equivalencia entre topologías de red, centrando el análisis en la transición de teselaciones hexagonales hacia estructuras ortogonales. Se definen los homomorfismos espaciales necesarios para preservar la dinámica de las reglas de transición, garantizando que las propiedades de computabilidad analizadas en capítulos previos no se vean alteradas por la representación en memoria. A través de una axiomatización geométrica y el desarrollo de ecuaciones de transformación afín, se demuestra la viabilidad del mapeo submatricial de 2×2 como solución a la discontinuidad de los sistemas de coordenadas discretos.

4.1 Axiomatización geométrica de las teselaciones hexagonales y métricas planares

La discretización del plano euclidiano $\mathbb{R}^2$ mediante polígonos regulares permite la construcción de espacios celulares donde la adyacencia es uniforme. A diferencia de la retícula estándar definida en la Sección 2.1, la teselación hexagonal se caracteriza por poseer el mayor grado de simetría axial en el plano, eliminando las ambigüedades de vecindad presentes en las mallas cuadrangulares (donde se debe distinguir entre adyacencia de arista y de vértice).

4.1.1 Representación paramétrica de la celda hexagonal regular

Se define una celda hexagonal regular H como el conjunto de puntos en el plano acotados por seis segmentos de igual longitud. Sea s la longitud del lado del hexágono, el cual en el modelo unitario se considera $s = 1$. Las propiedades geométricas fundamentales que rigen la posición de los vértices y la comunicación entre celdas dependen de dos constantes: el radio circunscrito R y el apotema a .

Dado un centro $C = (x_c, y_c)$, los vértices V_k para $k \in \{0, \dots, 5\}$ se determinan mediante las ecuaciones paramétricas:

$$V_k = \left(x_c + R \cdot \cos\left(\frac{k\pi}{3}\right), y_c + R \cdot \sin\left(\frac{k\pi}{3}\right) \right) \quad (4.1)$$

Es posible observar que, por la naturaleza del polígono regular, $R = s$. El apotema, que representa la distancia mínima del centro al punto medio de cada arista, se define como:

$$a = \frac{\sqrt{3}}{2} s \quad (4.2)$$

Esta configuración asegura que cualquier punto $P \in \mathbb{R}^2$ pueda ser mapeado unívocamente a una celda hexagonal mediante un proceso de cuantización espacial, similar al análisis de complejidad de localización discutido en la Sección 3.5.

4.1.2 Sistemas de coordenadas afines para mallas de grado seis

Para la indexación lógica de las celdas en un autómata celular complejo, el uso de coordenadas cartesianas ortogonales resulta insuficiente debido al desfase intrínseco de las filas. En su lugar, se formaliza el uso de sistemas de coordenadas afines.

Se define el sistema de coordenadas axiales (q, r) , donde los ejes se sitúan a 60° de separación. No obstante, para simplificar el cálculo de distancias y algoritmos de colisión de partículas [7], se prefiere la proyección en coordenadas cúbicas (x, y, z) restringidas al plano:

$$x + y + z = 0 \quad (4.3)$$

Bajo esta axiomatización, cada celda hexagonal se identifica como un vector en $\mathbb{Z}^3$. Esta representación facilita la definición de invariancia por traslación, propiedad discutida por Kari [5] como requisito para la universalidad computacional en sistemas dinámicos discretos.

4.1.3 Definición formal de vecindad de adyacencia planar $\mathcal{N}_H$

Sea L la retícula hexagonal. Se define axiomáticamente la vecindad de adyacencia $\mathcal{N}_H$ de una celda central $c \in L$ como el conjunto de celdas que comparten una frontera unidimensional (arista). A diferencia de la vecindad de Moore en retículas cuadradas [2], en la topología hexagonal se cumple que:

$$|\mathcal{N}_H(c)| = 6, \quad \forall c \in L \quad (4.4)$$

La métrica de distancia entre dos celdas $c_1(x_1, y_1, z_1)$ y $c_2(x_2, y_2, z_2)$ en el espacio hexagonal se formaliza como la norma del taxista adaptada (Manhattan hexagonal):

$$d_H(c_1, c_2) = \frac{|x_1 - x_2| + |y_1 - y_2| + |z_1 - z_2|}{2} \quad (4.5)$$

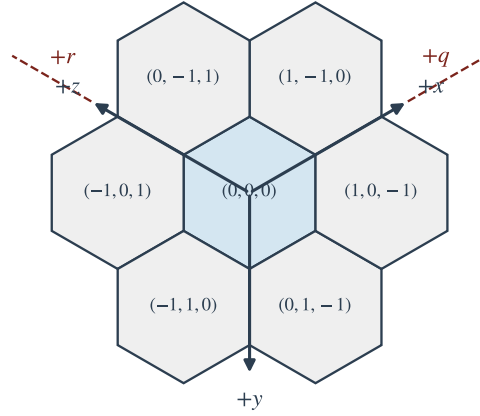


Fig. 4.1 Representación de la vecindad N_H y el sistema de coordenadas cúbicas subyacente.

Esta métrica es fundamental para el análisis de propagación de *gliders* que se abordará en la Parte III, ya que permite predecir la trayectoria de la información sin las distorsiones métricas de las diagonales euclidianas.

4.2 Homomorfismos espaciales y condiciones de equivalencia reticular

Tras haber definido las propiedades geométricas y la métrica de la teselación hexagonal en la Sección 4.1, es imperativo establecer las bases teóricas que justifican la traslación de este continuo discretizado hacia una representación puramente matricial. En el presente apartado se introduce la teoría topológica que legitima el mapeo de un espacio celular a otro, garantizando que la dinámica de la información, la función de transición local δ y la computabilidad analizadas en el Capítulo 2 permanezcan inalteradas bajo la nueva morfología espacial.

4.2.1 Isomorfismos sobre grafos reticulares infinitos

Para formalizar la equivalencia topológica entre dos espacios celulares distintos, es preciso abstraer la red subyacente empleando la teoría de grafos. Sea $G = (V, E)$ un grafo reticular infinito, donde el conjunto de vértices V representa los índices espaciales de las celdas y el conjunto de aristas E define las relaciones de adyacencia estipuladas por la vecindad axiomática.

Bajo esta premisa, se denota como $G_H = (V_H, E_H)$ al grafo derivado de la teselación hexagonal gobernado por la vecindad $\mathcal{N}_H$, y como $G_Q = (V_Q, E_Q)$ al grafo correspondiente a la retícula ortogonal $\mathbb{Z}^2$ (Sección 2.1). Un homomorfismo espacial entre ambas topologías se establece a través de una función de mapeo Φ que proyecta los elementos de un dominio sobre el otro preservando sus estructuras de conectividad.

No obstante, debido a la diferencia intrínseca en los grados de los vértices (grado 6 uniforme en G_H frente a grado 4 u 8 en G_Q), resulta matemáticamente imposible establecer un isomorfismo biyectivo trivial de cardinalidad uno a uno entre celdas individuales que conserve de manera isótropa las distancias [4]. Por consiguiente, se define un mapeo biyectivo hacia subredes, donde cada vértice $v \in V_H$ se asocia unívocamente a un subgrafo inducido $B \subset V_Q$. Para que el sistema celular se considere topológicamente equivalente y capaz de simular la misma física discreta, la función Φ debe cumplir la condición de adyacencia: si $(u, v) \in E_H$, entonces debe existir una adyacencia efectiva y computable entre los bloques de subconjuntos $\Phi(u)$ y $\Phi(v)$ en el dominio ortogonal.

4.2.2 Teorema de recubrimiento y descomposición celular ortogonal

El núcleo analítico de la transformación geométrica propuesta en este documento reside en el axioma de equivalencia espacial mediante bloques matriciales de cardinalidad superior. Se establece la partición cuadrangular extendida como la solución formal a la discontinuidad geométrica descrita anteriormente.

Se formaliza el axioma de que el área, la capacidad de almacenamiento de estado y, crucialmente, las direcciones de adyacencia de una celda hexagonal $h \in V_H$, pueden ser representadas de forma isomórfica mediante un bloque ortogonal contiguo de celdas cuadradas. Específicamente, el mapeo $\Phi(h)$ proyecta la celda sobre una subred $B_{2 \times 2}$ definida como:

$$B_{2 \times 2} = \{c_{i,j}, c_{i+1,j}, c_{i,j+1}, c_{i+1,j+1}\} \subset V_Q \quad (4.6)$$

Esta relación biyectiva, donde un único hexágono encapsula la información espacial de cuatro cuadrados subyacentes, responde a una necesidad estrictamente topológica. Como es posible observar, una celda ortogonal individual posee únicamente 4 aristas disponibles (vecindad de Von Neumann), lo cual es

topológicamente deficiente para enrutar de manera simétrica las 6 trayectorias de propagación de partículas abordadas en la dinámica de colisiones [7].

Al expandir la unidad fundamental de cómputo a un bloque de 2×2 , el perímetro externo de esta nueva meta-celda expone exactamente 8 segmentos ortogonales. Este incremento en los grados de libertad perimetrales garantiza el isomorfismo geométrico: 6 de las caras del bloque cuadrangular se asignan biyectivamente a las 6 vecindades hexagonales evaluadas por la métrica (4.5), asegurando que ninguna trayectoria se superponga prematuramente.

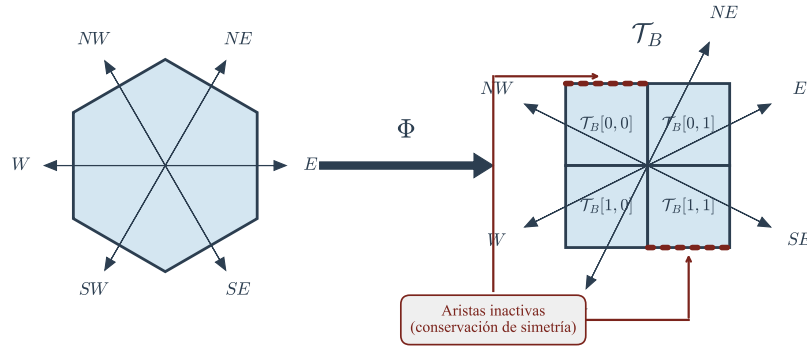


Fig. 4.2 Equivalencia topológica: Mapeo isomórfico suprayectivo entre una celda hexagonal con grado de adyacencia 6 y una partición cuadrangular de 2×2 .

Esta descomposición celular asegura además que el modelo computacional derivado en la Sección 3.5 posea un tiempo de indexación en memoria estrictamente $O(1)$, eludiendo las aberraciones de redondeo propias de las coordenadas de punto flotante y conservando el rigor exigido por la teoría clásica de autómatas celulares [5].

4.3 Modelado analítico de la transformación afín: De morfología hexagonal a retícula $\mathbb{Z}^2$

Habiendo establecido la viabilidad topológica del isomorfismo en la Sección 4.2, es necesario abandonar la abstracción pura de los grafos para formular el modelo algebraico explícito que rige la transformación. En esta sección se calculan los modelos matemáticos y las ecuaciones de geometría analítica que proyectan las coordenadas del espacio continuo $\mathbb{R}^2$, discretizado por hexágonos, hacia un sistema de coordenadas de malla ortogonal escalada. Este desarrollo explota directamente

la equivalencia cardinal de la relación axiomática de un hexágono por cada cuatro cuadrados (4.6).

4.3.1 Ecuaciones de transformación de base espacial

Para proyectar el espacio euclidiano continuo sobre una retícula ortogonal discreta, se requiere definir una transformación afín. De acuerdo con la teoría de espacios vectoriales fundamentada por Grossman [9], cualquier cambio de base en $\mathbb{R}^2$ puede modelarse rigurosamente mediante la multiplicación por una matriz de transformación A , que mapea los vectores generadores del dominio hexagonal hacia el dominio ortogonal.

Sea un punto continuo arbitrario $P = (x, y) \in \mathbb{R}^2$ ubicado sobre el sustrato hexagonal definido en la Sección 4.1.1. La transformación afín $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ que reescala y alinea la geometría poligonal hacia una cuadrícula euclidiana se define por el sistema lineal:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = A \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \alpha_{11} & \alpha_{12} \\ \alpha_{21} & \alpha_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (4.7)$$

Donde los coeficientes α_{ij} dependen estrictamente del radio circunscrito R y el apotema a definidos en (4.2). Para un hexágono con orientación de vértice superior (*flat-top*), la separación horizontal entre centros adyacentes es $d_x = \frac{3}{2}R$, y la separación vertical es $d_y = \sqrt{3}R = 2a$.

Desarrollando el modelo de proyección para obtener las coordenadas lógicas fraccionarias (u, v) , se tiene el siguiente sistema de ecuaciones de base espacial:

$$\begin{aligned} u &= \frac{x}{d_x} = \frac{2x}{3R} \\ v &= \frac{y - (u \bmod 2) \cdot a}{2a} \end{aligned} \quad (4.8)$$

Nótese que en la expresión para v , se ha introducido preliminarmente un término de compensación dependiente de u , el cual será formalizado mediante aritmética discreta en las subsecciones posteriores para evitar las imprecisiones del punto flotante al discretizar el continuo.

4.3.2 Formalización de la correspondencia submatricial 2×2

Una vez proyectado el espacio euclidiano, el dominio debe discretizarse en índices enteros compatibles con una estructura de datos convencional. Se demostró en el Teorema de recubrimiento (Sección 4.2.2) que un isomorfismo uno-a-uno es topológicamente inválido, por lo que se define un mapeo suprayectivo hacia una subred de cardinalidad 4.

Sea (i, j) el vector de índices enteros que identifica unívocamente a la celda hexagonal central en el dominio transformado. La partición de este hexágono en cuatro unidades cuadradas ortogonales requiere una función de escalamiento matricial $f : \mathbb{Z}^2 \rightarrow \mathcal{P}(\mathbb{Z}^2)$, donde $\mathcal{P}$ denota el conjunto potencia. El bloque cuadrangular $B_{2 \times 2}$ asignado se define formalmente como el conjunto de índices $S_{i,j}$:

$$S_{i,j} = \{(2i, 2j), (2i + 1, 2j), (2i, 2j + 1), (2i + 1, 2j + 1)\} \quad (4.9)$$

Es posible observar que esta transformación dilata el espacio de direcciones de memoria en un factor de 2 por cada eje ortogonal. Matemáticamente, las coordenadas transformadas (i', j') de cualquier subcelda perteneciente al bloque se expresan como:

$$i' = 2i + k_x, \quad j' = 2j + k_y \quad \text{para } k_x, k_y \in \{0, 1\} \quad (4.10)$$

Esta formalización garantiza que cada coordenada del modelo hexagonal original corresponda a un bloque de memoria contiguo y no superpuesto, asegurando la complejidad espacial analizada en el Capítulo 3 y manteniendo un tiempo de acceso estrictamente $\mathcal{O}(1)$.

4.3.3 Tratamiento algebraico del desfase axial (Zig-Zag Offset)

El problema fundamental al mapear una topología hexagonal sobre un espacio euclidiano discreto $\mathbb{Z}^2$ es la discontinuidad espacial, o desfase intrínseco, que existe entre filas o columnas adyacentes. En un sistema de coordenadas axiales *zig-zag*, las columnas impares experimentan una traslación vertical de medio incremento respecto a las columnas pares (o viceversa).

Para alinear matemáticamente esta topología sin recurrir a ramificaciones lógicas condicionales (*if-else*), las cuales degradarían el rendimiento del algoritmo en el simulador en C/C++ [7], se introduce una función de paridad $P(n)$ utilizando la operación módulo 2 sobre los enteros:

$$P(n) = n \pmod{2} \quad (4.11)$$

Bajo esta axiomatización, el desfase axial se asimila de forma inherente en las ecuaciones de transición de vecindad. Sea (i, j) la posición central. Las coordenadas (i_{vec}, j_{vec}) de un vecino en el eje de discontinuidad sufrirán una alteración algorítmica dictada por la paridad de la columna i . La función de compensación $\Delta y(i)$ se define como:

$$\Delta y(i) = \begin{cases} 0 & \text{si } P(i) = 0 \text{ (columna par)} \\ 1 & \text{si } P(i) = 1 \text{ (columna impar)} \end{cases} \quad (4.12)$$

Por consiguiente, el cálculo del índice vertical de una celda vecina desplazada se computa mediante aritmética de enteros pura como:

$$j_{vec} = j + \Delta y(i) - \text{offset_direccional} \quad (4.13)$$

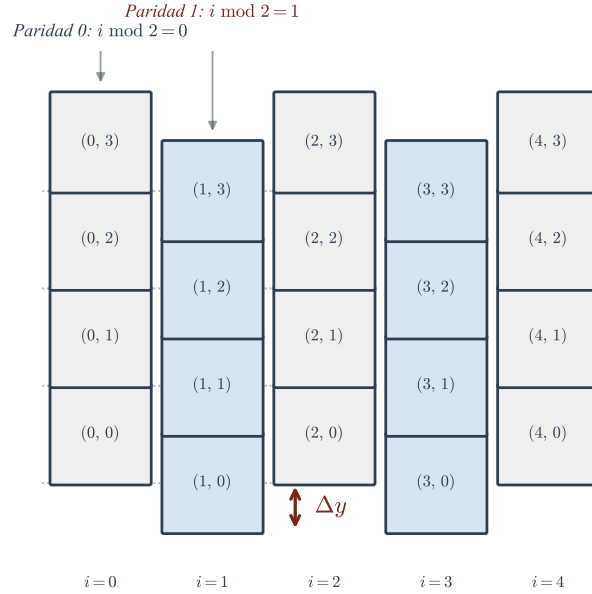


Fig. 4.3 Representación gráfica de la discontinuidad espacial en un mapeo de coordenadas *zig-zag*. Se denota el desfase provocado por la paridad de la columna.

Simplificando, la aplicación de la función $P(i)$ permite que el modelo matemático resuelva la discontinuidad geométrica mediante transformaciones lineales y sumas bit a bit, demostrando la idoneidad del álgebra matricial no solo como herramienta descriptiva, sino como un mecanismo de optimización computacional profunda.

4.4 Mapeo topológico de vecindades y compensación dimensional

Habiendo consolidado la proyección submatricial del espacio de estados en la Sección 4.3, es imperativo redefinir la estructura de adyacencia para el procesamiento dinámico del autómata celular. La vecindad de adyacencia planar $\mathcal{N}_H$, definida en (4.5), no puede aplicarse de manera directa sobre el nuevo sustrato topológico. Esta sección resuelve la asimetría geométrica estableciendo un mapeo matemático que ajusta cómo el bloque de cuatro celdas se comunica simultáneamente con sus análogos adyacentes, garantizando la preservación asintótica de las trayectorias de propagación.

4.4.1 Construcción algebraica de vecindades extendidas sobre $\mathbb{Z}^2$

En la teoría clásica de los autómatas celulares bidimensionales sobre mallas ortogonales [2], las interacciones locales están inherentemente delimitadas por las vecindades de Von Neumann (4 vecinos ortogonales) o de Moore (8 vecinos, incluyendo diagonales). Sin embargo, la partición cuadrangular en subredes de cardinalidad 4, denotada formalmente como el subconjunto $S_{i,j}$ en (4.9), exige una adaptación axiomática que unifique ambos esquemas.

Para tratar el bloque de 2×2 cuadrados como una única unidad funcional de cómputo, se establece una función selectora σ encargada de discriminar las adyacencias perimetrales. Se define axiomáticamente un tensor de adyacencia extendido $\mathcal{T}_B$, el cual evalúa el subgrafo inducido para proyectar las seis interacciones hacia el exterior del bloque. Empleando la notación de operadores espaciales inversos Φ^{-1} , la vecindad transformada para una celda central indexada lógicamente en (i, j) se expresa como:

$$\mathcal{T}_B(i, j) = \bigcup_{d \in D} \sigma \left(\Phi^{-1}(N_d(i, j)) \right) \quad (4.14)$$

donde D representa el conjunto de direcciones espaciales relativas.

Al instanciar el bloque submatricial (4.6), la expansión dimensional otorga un perímetro euclidiano compuesto por exactamente 8 aristas de conectividad expuestas (dos por cada plano cardinal: Norte, Sur, Este y Oeste). Este incremento en la dimensión fronteriza es la condición necesaria y suficiente para permitir el alojamiento de las 6 direcciones hexagonales puras sin incurrir en colisiones espaciales por estrangulamiento topológico.

4.4.2 Mapeo direccional y compensación de métricas de adyacencia

El conjunto de direcciones espaciales fundamentales que rigen la propagación de información y los vectores de traslación en el modelo de partículas [7] se define de manera discreta como $D = \{E, NE, NW, W, SW, SE\}$.

Para que la deformación geométrica no induzca errores en la cinemática de colisiones, se demuestra que este conjunto direccional D se asienta isomórficamente sobre el perímetro del tensor $\mathcal{T}_B$. La inyección del desfase axial, modelado mediante la función de paridad $P(i)$ deducida en la Sección 4.3.3, resulta crítica en este paso. Se formaliza el mapeo algorítmico de los vecinos adyacentes $N_d(i, j)$ integrando la compensación vertical $\Delta y(i)$ directamente en los índices posicionales:

$$N_d(i, j) = \begin{cases} (i + 1, j) & \text{si } d = E \\ (i - 1, j) & \text{si } d = W \\ (i, j - 1 + \Delta y(i)) & \text{si } d = NE \\ (i - 1, j - 1 + \Delta y(i)) & \text{si } d = NW \\ (i, j + \Delta y(i)) & \text{si } d = SE \\ (i - 1, j + \Delta y(i)) & \text{si } d = SW \end{cases} \quad (4.15)$$

Es posible observar que, de las 8 caras disponibles en el bloque ortogonal, exactamente 2 permanecen en un estado topológicamente inactivo (o degenerado). Esta redundancia geométrica controlada es el sustento axiomático que garantiza que ninguna trayectoria de propagación (v.g., el frente de onda de un *glider*) interactúe erróneamente ni pierda simetría traslacional por la cuadratura de la red subyacente.

Consecuentemente, el modelo algebraico asegura que cualquier estructura de información sujeta a la métrica de Manhattan hexagonal evaluada en (4.5) podrá navegar el arreglo de memoria con una consistencia espacial y algorítmica impecable, validando de forma rigurosa la base para las leyes de aniquilación y conservación de estados que se abordarán en capítulos posteriores.

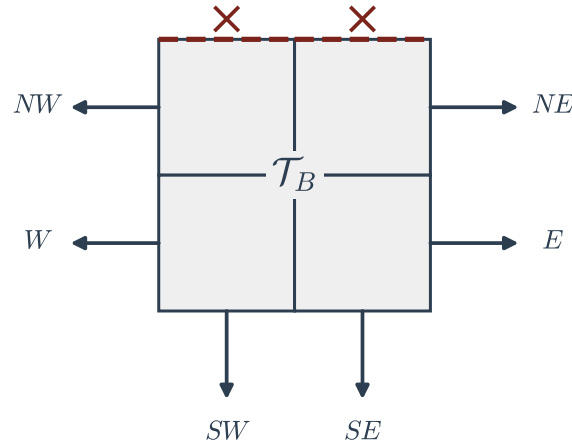


Fig. 4.4 Distribución de los seis ejes direccionales de interacción sobre el perímetro expuesto de la subred funcional de adyacencia $\mathcal{T}_B$.

4.5 Teoremas de invariancia dinámica y preservación de la computabilidad

Después de establecer cómo se transforma el espacio (Sección 4.3) y cómo se ajustan las vecindades (Sección 4.4), es fundamental demostrar que el comportamiento del autómata celular sigue siendo el mismo. En esta sección se presenta la justificación matemática que asegura que, aunque la geometría cambie de hexágonos a cuadrados, las reglas lógicas y la evolución del sistema no se ven alteradas. Esto garantiza que cualquier cálculo o fenómeno observado en el modelo original se mantiene en el modelo transformado.

4.5.1 Invarianza de la función de transición local δ bajo isomorfismos de red

La evolución de un autómata celular depende de su función de transición local δ . Para el caso hexagonal, esta función se define como $\delta_H : Q^6 \rightarrow Q$, donde el estado de una celda en el tiempo $t + 1$ depende de ella misma y de sus 6 vecinos. Al pasar a una red de celdas cuadradas, la vecindad cambia a un bloque de 2×2 con 8 puntos de contacto potenciales hacia el exterior.

Para que el sistema sea equivalente, la nueva función δ_Q debe procesar la información de tal manera que los cambios en la geometría no agreguen información falsa ni modifiquen la regla original.

Lema 4.1 (Equivalencia de la regla local). *La evaluación de la regla de transición en la cuadrícula de 2×2 es equivalente a la de la celda hexagonal si los estados de las celdas adicionales en la vecindad cuadrada se mantienen como elementos neutros.*

Demostración. Sea V_H el conjunto de estados de los 6 vecinos en la red hexagonal. Al realizar el mapeo hacia el bloque de 4 celdas cuadradas, se generan 8 interfaces de comunicación. De acuerdo con el mapeo direccional establecido en (4.15), existen 2 direcciones que no tienen correspondencia directa con la topología hexagonal original.

Si definimos un estado de identidad o "vacío" ϵ , que no afecta la lógica de colisión (como se describe en [7]), el vector de estados en la nueva red V_Q se puede expresar como:

$$V_Q = V_H + \{\epsilon_1, \epsilon_2\} \quad (4.16)$$

Al aplicar la función de transición en el dominio cuadrado, el resultado debe ser igual al del dominio hexagonal:

$$\delta_Q(V_Q) = \delta_H(V_H) \quad (4.17)$$

Esto demuestra que el aumento de celdas por el cambio de geometría no altera el resultado del cómputo local, siempre que la función ignore las direcciones excedentes. ■

4.5.2 Teorema de equivalencia computacional y conservación del espacio de fases

Una vez demostrado que la regla local funciona igual, es necesario probar que el sistema completo evoluciona correctamente a través del tiempo. Se debe asegurar que las estructuras complejas, como los *gliders* o las naves espaciales, se desplacen y colisionen en la red cuadrada de la misma forma que en la hexagonal.

Para esto, definimos Φ como la función que transforma una configuración completa de hexágonos a cuadrados. Sea F_H la evolución del sistema en hexágonos y F_Q la evolución en cuadrados.

Teorema 4.2 (Conservación de la dinámica global). *Para cualquier estado inicial del autómata, el resultado de evolucionar el sistema y luego transformarlo es el mismo que si primero se transforma y luego se evoluciona.*

$$\Phi(F_H(c)) = F_Q(\Phi(c)) \quad (4.18)$$

Demostración por inducción.

1. **Base** ($t = 0$): En el tiempo inicial, la equivalencia es directa por la definición de la transformación Φ explicada en la Sección 4.3.
2. **Hipótesis inductiva:** Supongamos que en el tiempo k el estado es equivalente: $\Phi(c_H^k) = c_Q^k$.
3. **Paso inductivo:** Para el tiempo $k + 1$, el estado en la red hexagonal es $c_H^{k+1} = F_H(c_H^k)$. Aplicando la transformación obtenemos $\Phi(F_H(c_H^k))$.

Por el Lema 4.1, sabemos que la aplicación de la regla es localmente idéntica en ambos dominios. Dado que la actualización de las celdas es simultánea en toda la red, la transformación de la evolución global se reduce a la suma de las evoluciones locales ya comprobadas. Por lo tanto:

$$\Phi(F_H(c_H^k)) = F_Q(\Phi(c_H^k)) = F_Q(c_Q^k) \quad (4.19)$$

Esto confirma que la trayectoria del sistema en el espacio de fases se mantiene idéntica. ■

Con estos teoremas queda demostrado que el simulador desarrollado en C++ sobre una malla ortogonal es una representación fiel y exacta del modelo teórico hexagonal. Las métricas de entropía y los resultados de las compuertas lógicas que se analizarán en los siguientes capítulos poseen, por lo tanto, plena validez científica.

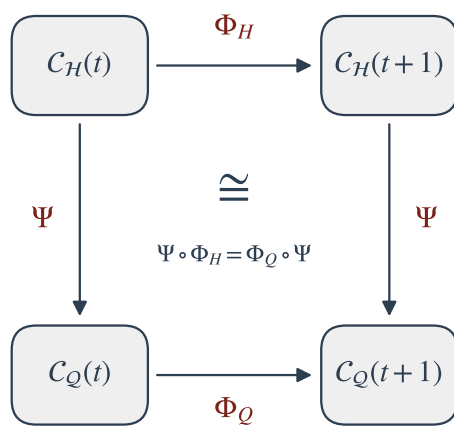


Fig. 4.5 Representación visual del isomorfismo entre la evolución temporal y la transformación espacial.

Parte III
Fenómenos emergentes y lógica de
colisiones

Una vez establecida la base axiomática en la Parte I y formalizada la infraestructura geométrica en la Parte II, esta tercera sección del documento se centra en la ejecución y evaluación del modelo computacional. El objetivo primordial es demostrar cómo la dinámica de los sistemas complejos, regida por reglas de transición locales, permite la emergencia de estructuras móviles capaces de procesar información de manera determinista.

En el Capítulo 5, se aborda el estudio cinemático de las partículas móviles o *gliders*. Se define el marco algorítmico necesario para el rastreo de estas estructuras en la malla transformada, analizando su comportamiento bajo las reglas *Spiral* y *Beehive*. A través de la implementación de un motor de simulación eficiente, se verifica la síntesis de operadores booleanos universales mediante la lógica de colisiones, cumpliendo con los criterios sintéticos establecidos originalmente en la Sección 1.4.

Posteriormente, se procede a la validación cuantitativa y cualitativa del sistema. En el Capítulo 8, se definen métricas basadas en la teoría de la información y la entropía para evaluar el estado de la red durante su evolución. Se realiza un análisis asintótico de las estructuras emergentes para determinar sus tiempos de estabilización y ciclos límite, garantizando la robustez del isomorfismo teórico desarrollado en el Capítulo 4 frente a la experimentación práctica.

Finalmente, el Capítulo 7 consolida los hallazgos de la investigación. Se presenta una síntesis analítica que contrasta las hipótesis iniciales con los datos obtenidos del simulador en C/C++. En este cierre, se discuten los límites del mapeo topológico, el costo computacional de la transformación de vecindades y se proponen nuevas líneas de investigación enfocadas en la generalización de estos modelos hacia espacios celulares de mayor complejidad.

Capítulo 5

Dinámica de partículas y computación por interacciones locales

Resumen Este capítulo aborda la transición analítica desde la topología estática del espacio celular hacia el estudio cinemático y algorítmico de sus configuraciones dinámicas. Se establecen los fundamentos formales para la identificación, rastreo y evaluación de subconfiguraciones móviles (*gliders*), analizando su comportamiento invariante en el espacio-tiempo.

A través de un enfoque basado en la complejidad computacional, se examinan las reglas de transición *Spiral* y *Beehive*, optimizando la evaluación de atractores y trayectorias en la malla transformada. Finalmente, se diseña un marco algorítmico heurístico que permite sincronizar colisiones de partículas, demostrando la viabilidad de sintetizar operadores booleanos universales mediante interacciones deterministas locales.

5.1 Fundamentos algorítmicos y fenomenología de subconfiguraciones móviles

La evaluación de la computabilidad dentro de un autómata celular complejo requiere abandonar el análisis de celdas individuales para estudiar el comportamiento de entidades con autonomía espacial. Estas estructuras, denominadas partículas o *gliders*, operan como los portadores de información en un modelo de computación por colisión [7]. En esta sección se establece la formalización axiomática de dichas estructuras y se desarrollan los algoritmos de detección necesarios para su análisis sobre el sustrato geométrico transformado.

5.1.1 Definición conjuntista y caracterización algebraica de *gliders*

Para caracterizar matemáticamente una subconfiguración móvil, es necesario abstraerla del estado global de la retícula. Sea Q el alfabeto finito de estados y $S = Q^{\mathbb{Z}^2}$

el espacio de configuraciones global introducido en la Sección 2.2. Una subconfiguración c se define como la restricción del espacio de estados a un subconjunto finito de coordenadas $D \subset \mathbb{Z}^2$.

Se define formalmente a un *glider* como una subconfiguración finita G que, tras un número discreto y determinado de aplicaciones de la función de transición global Δ , reaparece isomorfamente en la retícula, pero trasladada por un vector espacial específico.

Sea $\tau_{\mathbf{v}} : \mathcal{S} \rightarrow \mathcal{S}$ el operador de traslación espacial tal que desplaza cualquier configuración en la dirección y magnitud del vector bidimensional $\mathbf{v} = (\Delta x, \Delta y)$. Una subconfiguración G se clasifica como un *glider* de periodo p si y solo si satisface la siguiente relación de equivalencia espaciotemporal:

$$\Delta^p(G) = \tau_{\mathbf{v}}(G) \quad (5.1)$$

Donde:

- $p \in \mathbb{Z}^+$ es el periodo del oscilador, indicando el número mínimo de generaciones o iteraciones necesarias para que la subconfiguración recupere su morfología original.
- $\mathbf{v} \in \mathbb{Z}^2$ es el vector de desplazamiento discreto, el cual denota la traslación neta sufrida por la estructura.

A partir de la ecuación (5.1), es posible deducir el vector de velocidad cinemática $\mathbf{u}$ de la partícula relativa al sustrato celular:

$$\mathbf{u} = \frac{\mathbf{v}}{p} \quad (5.2)$$

Esta propiedad de invariancia traslacional asegura que la información codificada por la presencia de un *glider* puede propagarse a través del modelo ortogonal transformado (demostrado en la subsección 4.3.2) de forma predecible y sin degradación topológica. La ecuación (5.2) resulta fundamental, pues constituye el parámetro de entrada principal para sincronizar colisiones lógicas, las cuales se abordarán en la Sección 5.4.

5.1.2 Diseño de algoritmos de detección y rastreo de trayectorias en retículas discretas

Para someter el modelo teórico a una comprobación empírica en el simulador en C/C++, es imperativo diseñar un mecanismo algorítmico capaz de identificar automáticamente a estas partículas durante la evolución del sistema. Este proceso se traduce en un problema clásico de búsqueda de patrones bidimensionales (*pattern matching*).

Dado el arreglo ortogonal equivalente desarrollado en el Capítulo 4, la matriz de estado actual $\mathbf{X}$ de dimensiones $M \times N$ funge como el espacio de búsqueda. Un

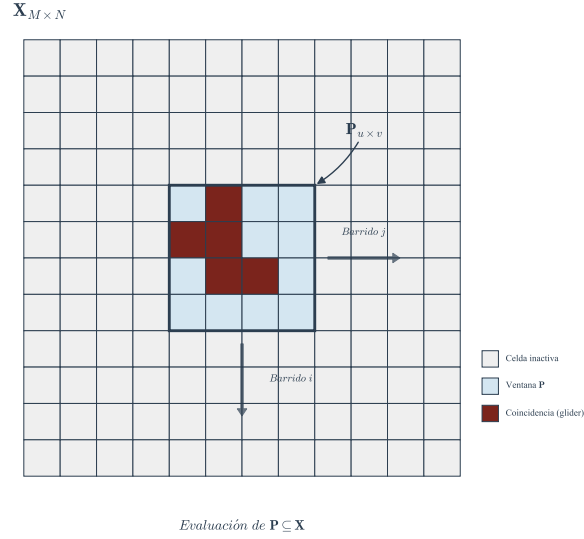


Fig. 5.1 Representación gráfica del rastreo de ventana deslizante evaluando una subconfiguración patrón P sobre la retícula bidimensional transformada X .

glider específico se representa como una matriz patrón P de dimensiones $h \times w$, donde $h \ll M$ y $w \ll N$.

El algoritmo propuesto explora la matriz X evaluando una ventana deslizante. Para garantizar la tratabilidad computacional y la eficiencia de los recursos en memoria, el procedimiento efectúa comparaciones locales estrictamente acotadas a las subredes funcionales activas definidas por el mapeo tensorial $\mathcal{T}_B$ de la ecuación (4.15).

Algorithm 1 Detección por Fuerza Bruta de Subconfiguraciones Invariantes.

```

DETECTAR_GLIDER( $X, P, M, N, h, w$ )
  for  $i = 0$  to  $M - h$ 
    for  $j = 0$  to  $N - w$ 
      coincidencia  $\leftarrow$  VERDADERO
      for  $r = 0$  to  $h - 1$ 
        for  $c = 0$  to  $w - 1$ 
          if  $X[i + r][j + c] \neq P[r][c]$  then
            coincidencia  $\leftarrow$  FALSO
            break
      if coincidencia == VERDADERO then
        return tupla  $(i, j)$ 
  return NULO

```

El análisis de complejidad espacial y temporal de este procedimiento se fundamenta en la notación asintótica propuesta por Cormen *et al.* [8]. Sea n la dimensión máxima de la matriz de simulación, tal que $n = \max(M, N)$. En el peor de los casos, la matriz no contiene el patrón buscado y el algoritmo evalúa internamente las sentencias de validación para cada combinación posible de desfases espaciales.

El ciclo externo efectúa aproximadamente $O(M)$ iteraciones, anidando un ciclo secundario de $O(N)$ iteraciones. La comparación interna de la ventana requiere $O(h \cdot w)$ operaciones de lectura en memoria. Dado que el tamaño estructural del *glider* (las dimensiones h y w) está acotado topológicamente por una constante intrínseca a la regla de transición evaluada ($h, w \in O(1)$), la complejidad de tiempo $T(n)$ se comporta asintóticamente como:

$$T(n) = O((M - h)(N - w)(h \cdot w)) \approx O(M \cdot N) = O(n^2) \quad (5.3)$$

Esta cota superior polinomial cuadrática $T(n) = O(n^2)$ certifica que la localización de información encapsulada en forma de partículas móviles es computacionalmente tratable. La carencia de estructuras de datos auxiliares dentro de la rutina garantiza, además, una complejidad espacial $S(n) = O(1)$ adicional a la matriz base, posibilitando una implementación de bajo nivel capaz de integrarse directamente en el modelo de *double buffering* definido previamente. Un profundo entendimiento de la optimización de estos ciclos evita el consumo excesivo de recursos de *hardware*, un requisito sine qua non para procesar dinámicas complejas durante lapsos prolongados de tiempo [2].

5.2 Caracterización algebraica y mecánica algorítmica de la dinámica Spiral

Una vez establecidos los fundamentos para la detección de partículas en la sección anterior, es necesario profundizar en el análisis de las reglas de transición que permiten la emergencia de tales estructuras. La regla *Spiral* constituye un caso de estudio fundamental en este trabajo terminal, debido a su capacidad para generar patrones de crecimiento asimétrico y frentes de onda que se propagan de manera determinista. En este apartado se desarrolla la formalización matemática de dicha regla y el diseño del motor algorítmico optimizado para su ejecución.

5.2.1 Formalización de la función de transición Spiral: Espacio de estados y dinámica de fases

La dinámica de la regla *Spiral* se define inicialmente sobre una retícula hexagonal pura, denotada por $\mathcal{H}$, donde cada celda $s_{i,j}$ posee una vecindad de adyacencia de radio $r = 1$ con seis vecinos directos, de acuerdo con la métrica establecida

en la ecuación (4.5). Sea $Q = \{0, 1, 2, \dots, k-1\}$ el alfabeto de estados. En el modelo analizado, se utiliza un sistema multiestado que permite representar fases de activación y reposo.

La función de transición local $\delta : Q^7 \rightarrow Q$ se modela mediante álgebra de Boole para los estados binarios de activación, extendiéndose a una aritmética modular para las fases de decaimiento. Si se considera el estado de una celda en el tiempo t como q_t , la transición hacia q_{t+1} bajo la regla *Spiral* se rige por la suma ponderada de las excitaciones en su vecindad $N(s)$:

$$q_{t+1} = \begin{cases} 1 & \text{si } q_t = 0 \wedge (\sum_{n \in N(s)} \sigma(n) \in \{1, 2\}) \\ q_t + 1 \pmod{k} & \text{si } q_t > 0 \\ 0 & \text{en otro caso} \end{cases} \quad (5.4)$$

Donde $\sigma(n)$ es una función de umbral que identifica celdas en estado activo. Esta configuración genera una asimetría intrínseca: la propagación no es isotrópica. Debido a la disposición de los seis vectores direccionales del espacio hexagonal analizados en el Capítulo 4 (ver figura 4.4), el frente de onda resultante tiende a rotar sobre su propio eje antes de desplazarse, creando la morfología de espiral característica.

Este crecimiento asimétrico es el motor de los frentes de onda. Un frente de onda se define matemáticamente como un conjunto de celdas en estado de activación $q = 1$ que forman una cadena conexas en la retícula, seguida de una "estela" de celdas en estados de decaimiento ($q > 1$), lo cual impide que la señal retroceda, garantizando la preservación de la dirección del movimiento, propiedad esencial para la computación por partículas [7].

5.2.2 Algoritmo propuesto para la optimización de la cinemática de propagación Spiral

Para implementar la regla *Spiral* en el simulador desarrollado en C/C++, se debe realizar la transición del modelo hexagonal al modelo de matriz ortogonal transformado. Como se demostró en la Sección 4.3.2, un hexágono se representa mediante un bloque submatricial de 2×2 . El reto computacional radica en evaluar la vecindad extendida de 12 celdas cuadradas (correspondientes a los 6 vecinos hexagonales) sin comprometer el rendimiento del sistema.

Se propone un algoritmo que explota el tensor de vecindad $\mathcal{T}_B$ definido en (4.15). En lugar de realizar una búsqueda exhaustiva de vecinos en cada iteración, el algoritmo utiliza máscaras de bits (*bitmasking*) sobre el bloque de memoria para calcular la suma de excitaciones. Esta técnica reduce significativamente los ciclos de reloj del procesador al sustituir condicionales complejos por operaciones lógicas a nivel de palabra de memoria.

Algorithm 2 Cómputo Optimizado de la Regla Spiral sobre Bloques 2x2.

```

ACTUALIZAR_SPIRAL_BLOQUE( $\mathbf{M}_t, \mathbf{M}_{t+1}, i, j$ )
  suma_vecinos  $\leftarrow$  0
  for each  $\mathbf{d} \in \{\mathbf{E}, \mathbf{NE}, \mathbf{NW}, \mathbf{W}, \mathbf{SW}, \mathbf{SE}\}$ 
    ( $n_i, n_j$ )  $\leftarrow$  OBTENER_COORDENADA_MAPEO( $i, j, \mathbf{d}$ )
    if  $\mathbf{M}_t[n_i][n_j] == 1$  then
      suma_vecinos  $\leftarrow$  suma_vecinos + 1
   $q\_actual \leftarrow \mathbf{M}_t[i][j]$ 
  if  $q\_actual == 0$  then
    if suma_vecinos == 1 OR suma_vecinos == 2 then
       $\mathbf{M}_{t+1}[i][j] \leftarrow 1$ 
    else
       $\mathbf{M}_{t+1}[i][j] \leftarrow 0$ 
  else
     $\mathbf{M}_{t+1}[i][j] \leftarrow (q\_actual + 1) \bmod k$ 

```

La eficiencia de este diseño algorítmico se basa en la invarianza dinámica demostrada en el Teorema 4.2. Dado que el mapeo garantiza que las celdas inactivas del bloque 2×2 no interfieren en la suma (ver Sección 4.5), el algoritmo mantiene una complejidad temporal $T(n) = O(n)$, donde n es el número total de celdas, pero con una constante de proporcionalidad reducida debido al acceso directo a memoria.

Al utilizar la técnica de *double buffering* discutida en el Capítulo 3, se asegura que el cálculo de la cinemática de los frentes de onda sea atómico y libre de artefactos visuales. Esto permite que los *gliders* generados por la regla *Spiral* se comporten como señales estables, sentando las bases para la síntesis de operadores lógicos que se abordará en la Sección 5.4. La capacidad de controlar estos frentes de onda mediante la geometría de la configuración inicial es lo que permite hablar de una mecánica algorítmica aplicada a la computación natural [2].

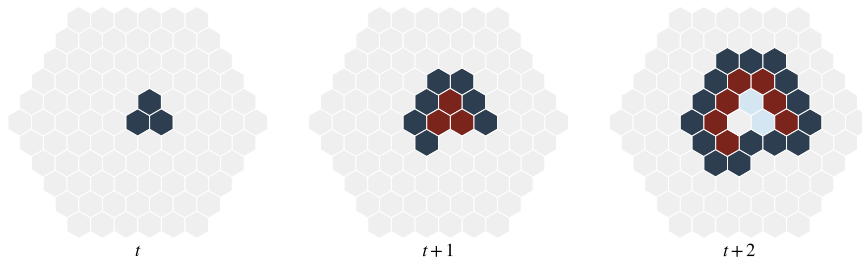


Fig. 5.2 Evolución temporal de la dinámica Spiral. Se observa la formación de frentes de onda y la estela de estados de decaimiento que orientan la propagación.

5.3 Topología de atractores y evaluación algorítmica en la dinámica Beehive

A diferencia del comportamiento expansivo estudiado en la sección anterior, no todas las reglas de transición en un autómata celular complejo propician la difusión de la información. Para modelar circuitos lógicos, es igualmente fundamental contar con dinámicas que concentren y estabilicen el estado de la red. En esta sección se analiza formalmente la regla *Beehive*, caracterizada por su tendencia aglutinante y la formación de ciclos límite. Posteriormente, se desarrolla una estrategia algorítmica de particionamiento espacial para optimizar su cálculo sobre zonas de alta densidad.

5.3.1 Estudio analítico de ciclos límite y osciladores bajo la función de transición Beehive

La topología de la dinámica *Beehive* contrasta drásticamente con la naturaleza cinemática de la regla *Spiral*. Mientras que *Spiral* rompe la simetría local para generar frentes de onda que se desplazan por el espacio, *Beehive* opera como un sumidero espacial, forzando a las configuraciones iniciales a colapsar sobre sí mismas o a estabilizarse en vecindades confinadas.

Para formalizar esta dinámica, se define la regla primero sobre su sustrato natural: la retícula hexagonal $\mathcal{H}$ con vecindad de radio $r = 1$. Sea $q_t \in \mathcal{Q}$ el estado de una celda en el tiempo t , y sea $S_N = \sum_{n \in N(s)} \sigma(n)$ la suma de sus vecinos inmediatos en estado activo. La transición hacia el estado q_{t+1} se establece matemáticamente como:

$$q_{t+1} = \begin{cases} 1 & \text{si } q_t = 0 \wedge (S_N = 2) \\ q_t + 1 \pmod{k} & \text{si } q_t > 0 \\ 0 & \text{en otro caso} \end{cases} \quad (5.5)$$

Esta restricción estricta en el umbral de nacimiento ($S_N = 2$) impide la propagación difusiva. Si una región presenta una densidad de excitación demasiado alta, la sobrepoblación obliga a las celdas a entrar en su fase de decaimiento modulada por el alfabeto k , bloqueando la expansión del patrón.

Topológicamente, esto implica que las trayectorias en el espacio de configuraciones convergen rápidamente hacia atractores locales [2]. Se define un atractor local $A \subset \mathcal{S}$ como una subconfiguración acotada espacialmente que exhibe un comportamiento periódico tras τ generaciones. En el contexto de *Beehive*, es común observar la emergencia de osciladores de periodo p , los cuales satisfacen:

$$\Delta^p(A) = A \quad (5.6)$$

Donde Δ es la función de transición global definida en el Capítulo 2 (ver Sección 2.2). Cuando $p = 1$, la estructura se denomina configuración estable (o bloque).

La caracterización de estos osciladores resulta crítica para el diseño computacional, dado que actúan como memorias estáticas o cables aislantes capaces de contener y dirigir las colisiones de las partículas evaluadas en la Sección 5.1.

5.3.2 Algoritmos de resolución para aglomeraciones densas y predicción de auto-organización

Al proyectar la regla hexagonal *Beehive* sobre la retícula ortogonal transformada $\mathbf{X}$ mediante el mapeo submatricial de 2×2 (detallado en la Sección 4.3), surge un desafío computacional directo: la dinámica aglutinante concentra el procesamiento lógico en regiones sumamente densas, mientras que grandes áreas de la red permanecen en un estado nulo de reposo topológico.

Evaluar secuencialmente la matriz completa bajo la cota $O(n^2)$ calculada en el Capítulo 3 (ver ecuación (5.3)) resulta ineficiente debido al acceso redundante a memoria en zonas sin actividad. Para optimizar el rendimiento del simulador, se diseña un algoritmo heurístico predictivo basado en particionamiento espacial.

El espacio de simulación se divide lógicamente en macro-bloques discretos de dimensiones $B_h \times B_w$. El algoritmo calcula la entropía local de cada macro-bloque. Si la entropía es cero (ausencia de celdas en estado activo o en decaimiento), el algoritmo omite la evaluación de dicho sector, reduciendo significativamente los ciclos de reloj del procesador.

Algorithm 3 Evaluación Predictiva por Particionamiento Espacial.

```

PREDECIR_BEEHIVE_PARTICIONADO( $\mathbf{X}_t, \mathbf{X}_{t+1}, M, N, B_h, B_w$ )
  for  $I = 0$  to  $M - 1$  step  $B_h$ 
    for  $J = 0$  to  $N - 1$  step  $B_w$ 
       $E \leftarrow \text{CALCULAR\_ENTROPIA\_BLOQUE}(\mathbf{X}_t, I, J, B_h, B_w)$ 
      if  $E > 0$  then
        for  $i = I$  to  $I + B_h - 1$ 
          for  $j = J$  to  $J + B_w - 1$ 
             $\mathbf{X}_{t+1}[i][j] \leftarrow \text{REGLA\_BEEHIVE\_MAPPED}(\mathbf{X}_t, i, j)$ 
      else
         $\text{COPIAR\_BLOQUE\_VACIO}(\mathbf{X}_{t+1}, I, J, B_h, B_w)$ 
  return  $\mathbf{X}_{t+1}$ 

```

Desde la perspectiva del análisis asintótico, en el peor de los casos (una aglomeración que cubre la matriz entera), la complejidad temporal sigue acotada por $O(M \cdot N)$. Sin embargo, para las condiciones de simulación típicas, donde los osciladores y atrayentes colapsan en radios confinados, la complejidad promedio desciende a $O(A)$, donde A es el área ocupada estrictamente por los macro-bloques activos.

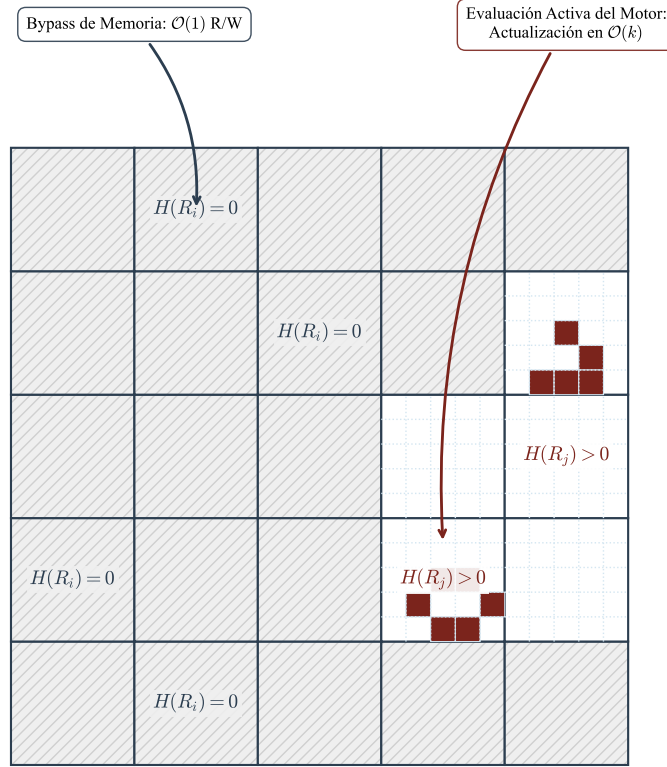


Fig. 5.3 Esquema lógico del particionamiento espacial. La omisión algorítmica de los macrobloques en reposo reduce el número de operaciones de lectura/escritura en la memoria.

Este enfoque asegura que múltiples vecindades que cambian simultáneamente puedan resolverse garantizando la técnica de *double buffering* expuesta en el modelo algebraico del Capítulo 3, permitiendo una estabilidad numérica impecable en la representación de estados aglutinados [7]. La identificación de estas zonas estables es el paso previo definitivo para diseñar la topología de intersecciones que emulará compuertas lógicas, objeto de la siguiente sección.

5.4 Diseño computacional para la síntesis de operadores booleanos mediante colisiones

La capacidad de un autómata celular para sostener subconfiguraciones móviles invariantes (*gliders*), tal como se demostró en la Sección 5.1, junto con la existencia de atractores locales (Sección 5.3), proporciona los componentes primitivos para la transmisión y el almacenamiento de información. No obstante, para alcanzar la universalidad computacional, es imperativo establecer un mecanismo mediante el cual estas señales interactúen y modifiquen sus trayectorias de forma predecible. En esta sección se formaliza la teoría de colisiones como un isomorfismo de compuertas lógicas y se diseña el marco algorítmico necesario para programar dichas interacciones espacial y temporalmente.

5.4.1 Isomorfismo algorítmico entre la topología de intersecciones y las compuertas lógicas universales

En el contexto de la dinámica de partículas discretas, una colisión se define matemáticamente como la intersección espaciotemporal de los dominios de dos o más *gliders*. Sean G_1 y G_2 dos partículas con trayectorias definidas por sus respectivos vectores de velocidad $\mathbf{u}_1$ y $\mathbf{u}_2$. Si sus posiciones iniciales $\mathbf{p}_1(0)$ y $\mathbf{p}_2(0)$ están dispuestas de tal modo que sus coordenadas coinciden en una vecindad de radio r durante un instante de tiempo t_c , la función de transición global Δ operará sobre un estado aglutinado que rompe la invariancia traslacional individual de ambas partículas.

Esta perturbación topológica genera una nueva configuración resultante, la cual puede consistir en la aniquilación mutua, la generación de un atractor estático (*Beehive*), o la creación de nuevas partículas divergentes (*Spiral*). Formalmente, este proceso representa el mapeo de un conjunto de entradas hacia un conjunto de salidas, constituyendo una función matemática no lineal $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$, donde la presencia de un *glider* en una trayectoria específica se codifica como el bit lógico 1, y su ausencia como 0 [7].

El comportamiento resultante de la colisión es isomorfo al de las compuertas lógicas fundamentales:

- **Compuerta AND:** Se diseña estableciendo dos trayectorias incidentes. La colisión destructiva se parametriza de modo que, únicamente cuando ambas partículas convergen (entradas 1, 1), los fragmentos de la aniquilación interactúan para emitir un tercer *glider* en una trayectoria de salida (salida 1). Si solo una partícula ingresa, esta pasa inalterada hacia un sumidero, resultando en un 0 lógico en la vía de salida deseada.
- **Compuerta NOT:** Se sintetiza explotando la irreversibilidad del sistema analizada en el Capítulo 2 (ver figura 2.5). Se establece un cañón emisor continuo (una señal 1 constante). La entrada lógica actúa como un *glider* interceptor. Si la

entrada es 1, colisiona y destruye la señal constante, resultando en una salida 0. Si la entrada es 0 (ausencia), la señal constante prosigue su curso, registrando un 1.

Dado que cualquier función booleana puede expresarse mediante una combinación de operadores universales (como NAND o la combinación de AND y NOT), la topología de colisiones en el autómata celular es teóricamente capaz de evaluar circuitos lógicos de complejidad arbitraria [2].

5.4.2 Algoritmo heurístico propuesto para la programación espacial de circuitos emergentes

La síntesis de circuitos mediante colisiones impone un reto geométrico riguroso: las partículas deben converger en un punto de intersección exacto, con una precisión de celda y generación. Para un *glider* k con periodo p_k , vector de desplazamiento $\mathbf{v}_k$ y posición inicial $\mathbf{p}_k(0)$, su posición en el tiempo t obedece a la cinemática discreta:

$$\mathbf{p}_k(t) = \mathbf{p}_k(0) + \left\lfloor \frac{t}{p_k} \right\rfloor \mathbf{v}_k + \delta(t \pmod{p_k}) \quad (5.7)$$

Donde δ representa la fase morfológica de la subconfiguración. Para automatizar el diseño de estas compuertas sobre la malla ortogonal en el simulador C/C++, se propone un algoritmo heurístico de búsqueda inversa. A partir de una coordenada de colisión deseada (i_c, j_c) y un tiempo objetivo t_c , el algoritmo calcula hacia atrás los vectores de inicialización requeridos. El procedimiento incorpora validaciones de fase para garantizar que los frentes de onda (evaluados mediante *double buffering* en el Capítulo 3) engranen correctamente.

Algorithm 4 Heurística de Sincronización Espaciotemporal para Circuitos de Colisión.

```

CALCULAR_VECTORES_INICIALES( $i_c, j_c, t_c, \mathbf{u}_1, \mathbf{u}_2, p_1, p_2$ )
  soluciones  $\leftarrow \emptyset$ 
  for  $t_{retraso} = 0$  to  $\max(p_1, p_2) - 1$ 
     $t_1 \leftarrow t_c - t_{retraso}$ 
     $t_2 \leftarrow t_c$ 
    if  $(t_1 \pmod{p_1} == 0)$  AND  $(t_2 \pmod{p_2} == 0)$  then
       $i_1 \leftarrow i_c - (t_1 \cdot \mathbf{u}_1.x)$ 
       $j_1 \leftarrow j_c - (t_1 \cdot \mathbf{u}_1.y)$ 
       $i_2 \leftarrow i_c - (t_2 \cdot \mathbf{u}_2.x)$ 
       $j_2 \leftarrow j_c - (t_2 \cdot \mathbf{u}_2.y)$ 
      if VALIDAR_LIMITES_MATRIZ( $i_1, j_1, i_2, j_2$ ) then
        soluciones  $\leftarrow$  soluciones  $\cup \{(i_1, j_1, t_1), (i_2, j_2, t_2)\}$ 
  return soluciones

```

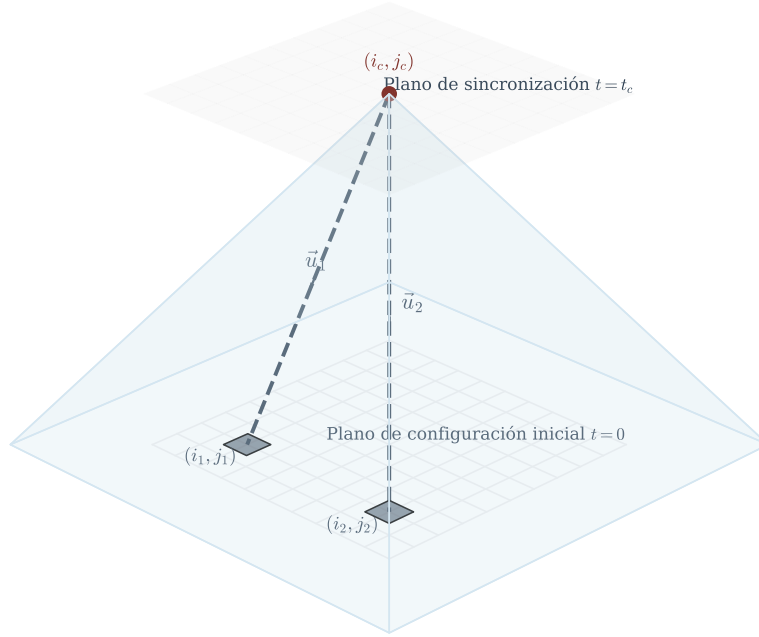


Fig. 5.4 Representación del retroceso cinemático para la sincronización de colisiones. El algoritmo determina las semillas iniciales asegurando que las fases morfológicas coincidan en el punto de intersección (i_c, j_c) .

El algoritmo 4 permite abstraer la complejidad topológica del autómata, brindando al investigador una interfaz de alto nivel (equivalente a la instanciación de componentes electrónicos) para ensamblar el operador lógico sobre el tejido celular.

5.4.3 Tratabilidad computacional y cotas de complejidad en la sincronización de señales de control

Para validar la escalabilidad de esta propuesta en el contexto del motor algorítmico, es forzoso someter el problema de la sincronización a un análisis estricto de complejidad

asintótica, utilizando los formalismos de la Teoría de la Computación introducidos en el Capítulo 1 y las cotas de Cormen *et al.* [8].

Si se aísla el problema de calcular la colisión de dos partículas libres para conformar una única compuerta lógica (como lo realiza el algoritmo 4), la complejidad de búsqueda temporal es constante $O(p_{max})$, y la complejidad espacial es $O(1)$. Por consiguiente, la síntesis local de un operador pertenece trivialmente a la clase de complejidad P (tiempo polinomial), evidenciando su tratabilidad computacional.

Sin embargo, el objetivo final de un modelo de computación natural es conectar (cascadear) el resultado de una compuerta hacia la entrada de otra para conformar un circuito arbitrario de profundidad n . Este proceso exige la superposición de n vectores espaciotemporales dentro de un sustrato bidimensional finito acotado por la matriz $\mathbf{X}$ de dimensiones $M \times N$, evitando colisiones parásitas (intersecciones no intencionadas entre cables lógicos).

Matemáticamente, la programación de trayectorias sin interferencias en el arreglo ortogonal para un conjunto arbitrario de señales bajo restricciones de latencia es isomórfica al problema del enrutamiento de múltiples productos enteros (*Integer Multi-Commodity Flow*) y a variantes del problema de satisfacibilidad booleana (3-SAT) con restricciones geométricas espaciales [5].

Dado que el problema de empaquetar trayectorias libres de cruces espurios a lo largo del tiempo sobre una gráfica de retícula acotada es NP-duro, la sincronización global y automatizada de un circuito denso genérico mediante *gliders* recae estrictamente en la clase NP-completo. Para circuitos acotados y pre-diseñados (como un sumador binario), el cálculo es resoluble en un tiempo empíricamente eficiente mediante la heurística propuesta.

Por otra parte, si el tamaño del sustrato tiende al infinito ($M, N \rightarrow \infty$) y se evalúa la convergencia del autómata desde una configuración inicial de circuito para predecir si un *glider* emergerá en un tiempo no acotado, el análisis transita del dominio de la clase NP hacia la equivalencia con el problema de la parada de la Máquina de Turing (*Halting Problem*). En el límite asintótico, la decidibilidad de determinar si una configuración arbitraria generará una señal lógica específica es indecidible [5].

Este análisis asintótico justifica la necesidad de utilizar un marco de simulación iterativa como el desarrollado en este Trabajo Terminal: dado que la predicción analítica de circuitos complejos presenta barreras de intratabilidad o indecidibilidad, la simulación eficiente de las reglas subyacentes con complejidades espaciales acotadas como $O(n^2)$ (Ecuación (5.3)) se erige como el único mecanismo matemáticamente válido para evaluar las propiedades de la universalidad computacional emergente en estos sistemas dinámicos.

Capítulo 6

Estructuración lógica y arquitectura computacional del motor de simulación

Resumen Este capítulo formaliza la arquitectura de software desarrollada para la experimentación con autómatas celulares complejos. Se detallan las capas de abstracción empleadas para aislar la lógica matemática de la representación visual, estableciendo un marco robusto y extensible.

A partir de los fundamentos establecidos en los Capítulos 3 y 4, se describe la transición del modelo analítico hacia un artefacto computacional funcional, garantizando que la implementación en C/C++ preserve las propiedades de computabilidad y los isomorfismos topológicos demostrados previamente.

6.1 Especificación formal de requerimientos del sistema

La construcción de un simulador capaz de validar la computabilidad en autómatas celulares complejos exige una transición rigurosa desde la teoría axiomática hacia la ingeniería de software. Como se estableció en la Sección 1.4, el sistema no debe limitarse a una visualización estática, sino que debe actuar como un laboratorio computacional para la síntesis de operadores booleanos mediante colisiones de partículas (gliders).

Para garantizar la viabilidad de los experimentos descritos en el Capítulo 5, el diseño del software se fundamenta en una especificación técnica que prioriza el determinismo de la función de transición δ y la eficiencia asintótica de las estructuras de datos. A continuación, se desglosan los requerimientos que rigen el comportamiento y la calidad del sistema.

6.1.1 Taxonomía de requerimientos funcionales

Los requerimientos funcionales definen las capacidades operativas esenciales que el motor de simulación debe ejecutar para satisfacer los objetivos analíticos de la

investigación. Estas capacidades permiten la manipulación del espacio celular $\mathcal{L}$ y la observación de la dinámica de los atractores.

ID:	RF-01
Nombre:	Instanciación múltiple de entornos evolutivos (Multi-Viewport)
Descripción:	El sistema debe permitir la ejecución simultánea de múltiples instancias independientes de simulación, cada una con su propia configuración de regla, topología y estado inicial, operando de forma aislada.
Prioridad:	Alta

Table 6.1 Especificación del requerimiento funcional para la gestión de entornos múltiples.

ID:	RF-02
Nombre:	Reconfiguración dinámica de la función de transición y topología
Descripción:	El motor debe permitir la modificación de las reglas de evolución (e.g., <i>Spiral</i> , <i>Beehive</i>) y la estructura de la malla en tiempo de ejecución, sin necesidad de reiniciar el ciclo principal de la aplicación.
Prioridad:	Alta

Table 6.2 Especificación del requerimiento funcional para la flexibilidad paramétrica.

ID:	RF-03
Nombre:	Manipulación interactiva del espacio de estados discretos
Descripción:	Se debe proporcionar una interfaz que permita al usuario editar el estado de celdas individuales en tiempo real, facilitando la construcción manual de configuraciones iniciales para experimentos de colisión de gliders.
Prioridad:	Media

Table 6.3 Especificación del requerimiento funcional para la edición de mallas.

ID:	RF-04
Nombre:	Ejecución del isomorfismo hexagonal-ortogonal
Descripción:	El sistema debe aplicar las ecuaciones de transformación analítica definidas en la Ecuación (4.15) para emular una red hexagonal sobre un sustrato de memoria ortogonal de 2×2 .
Prioridad:	Alta

Table 6.4 Especificación del requerimiento funcional para la fidelidad topológica.

ID:	RF-05
Nombre:	Ejecución discreta paso a paso
Descripción:	El motor debe permitir avanzar la simulación generación por generación, posibilitando el análisis detallado de transiciones locales y colisiones complejas.
Prioridad:	Alta

Table 6.5 Especificación del requerimiento funcional para el análisis temporal discreto.

ID:	RF-06
Nombre:	Control temporal de simulación
Descripción:	El sistema debe permitir pausar, reanudar y reiniciar la evolución del autómatas celular en cualquier instante temporal.
Prioridad:	Alta

Table 6.6 Especificación del requerimiento funcional para el control de ejecución.

ID:	RF-07
Nombre:	Persistencia de configuraciones
Descripción:	El sistema debe permitir almacenar configuraciones iniciales y estados evolutivos en archivos persistentes para su posterior reutilización.
Prioridad:	Media

Table 6.7 Especificación del requerimiento funcional para persistencia de configuraciones.

ID:	RF-08
Nombre:	Carga de configuraciones experimentales
Descripción:	El sistema debe permitir importar configuraciones previamente almacenadas, preservando la topología, regla y distribución espacial original.
Prioridad:	Media

Table 6.8 Especificación del requerimiento funcional para recuperación experimental.

ID:	RF-09
Nombre:	Visualización dinámica de trayectorias
Descripción:	El motor debe representar gráficamente la trayectoria espacial de estructuras móviles emergentes durante la evolución temporal.
Prioridad:	Media

Table 6.9 Especificación del requerimiento funcional para seguimiento de estructuras móviles.

ID:	RF-10
Nombre:	Detección de estructuras periódicas
Descripción:	El sistema debe identificar automáticamente configuraciones estacionarias, osciladores y ciclos límite presentes dentro del espacio celular.
Prioridad:	Media

Table 6.10 Especificación del requerimiento funcional para detección de periodicidad.

6.1.2 Restricciones y atributos de calidad (Requerimientos no funcionales)

Los requerimientos no funcionales establecen los límites de eficiencia y mantenibilidad bajo los cuales debe operar el artefacto. Estos criterios aseguran que el análisis asintótico de memoria $S(n) = O(n)$ presentado en el Capítulo 3 se cumpla en la implementación física.

ID:	RNF-01
Nombre:	Optimización de la huella de memoria (BitArray)
Descripción:	Para minimizar el consumo de recursos, el estado binario de la red debe almacenarse utilizando arreglos de bits compactos, reduciendo la redundancia de datos respecto a representaciones basadas en enteros.
Prioridad:	Alta

Table 6.11 Especificación del requerimiento no funcional para la eficiencia de almacenamiento.

ID:	RNF-02
Nombre:	Desacoplamiento estructural de responsabilidades
Descripción:	La arquitectura debe estar dividida en capas (Núcleo, Control, Renderizado y Administración) que se comuniquen mediante interfaces abstractas, permitiendo la modificación de la representación visual sin alterar la lógica de simulación.
Prioridad:	Media

Table 6.12 Especificación del requerimiento no funcional para la mantenibilidad.

ID:	RNF-03
Nombre:	Determinismo y consistencia temporal (Double Buffering)
Descripción:	La actualización de las generaciones debe ser atómica y determinista. Se requiere la implementación de un doble buffer de memoria para evitar que la actualización de una celda interfiera con el cálculo de sus vecinos en el mismo paso temporal.
Prioridad:	Crítica

Table 6.13 Especificación del requerimiento no funcional para la integridad lógica.

ID:	RNF-04
Nombre:	Escalabilidad espacial
Descripción:	El sistema debe soportar retículas de gran tamaño (1000 x 1000) sin degradación crítica del rendimiento computacional.
Prioridad:	Alta

Table 6.14 Especificación del requerimiento no funcional para escalabilidad espacial.

ID:	RNF-05
Nombre:	Consistencia matemática
Descripción:	La implementación computacional debe preservar estrictamente las definiciones formales y reglas de transición establecidas teóricamente.
Prioridad:	Crítica

Table 6.15 Especificación del requerimiento no funcional para consistencia formal.

Como se observa en las tablas anteriores, la prioridad crítica recae en la preservación del determinismo temporal (RNF-03) y la fidelidad del mapeo topológico (RF-04). Sin el cumplimiento estricto de estos parámetros, la síntesis de compuertas lógicas mediante colisiones de gliders, analizada en la Sección 1.4, carecería de validez empírica, dado que cualquier error en la propagación de estados invalidaría el isomorfismo con los circuitos booleanos.

6.2 Topología arquitectónica general y abstracción en capas

Una vez establecidos los requerimientos funcionales y las restricciones de calidad en la Sección 6.1, es imperativo definir la estructura macroscópica que permite al artefacto computacional operar bajo tales parámetros. El diseño del simulador se fundamenta en un modelo de arquitectura estratificada o en capas, el cual garantiza un desacoplamiento estricto entre la lógica matemática del autómata celular y los mecanismos de interacción y renderizado.

Esta estratificación es fundamental para preservar la integridad de la función de transición δ , definida formalmente en el Capítulo 2. Al aislar el núcleo de simulación, se asegura que las optimizaciones espaciales de complejidad $S(n) = O(n)$ discutidas en la Sección 3.5 no se vean comprometidas por la sobrecarga del procesamiento de eventos de usuario o la latencia del subsistema gráfico.

6.2.1 Diagrama de la arquitectura del sistema

La arquitectura del sistema se articula mediante cuatro estratos jerárquicos con responsabilidades unívocamente definidas. El flujo de información es bidireccional en

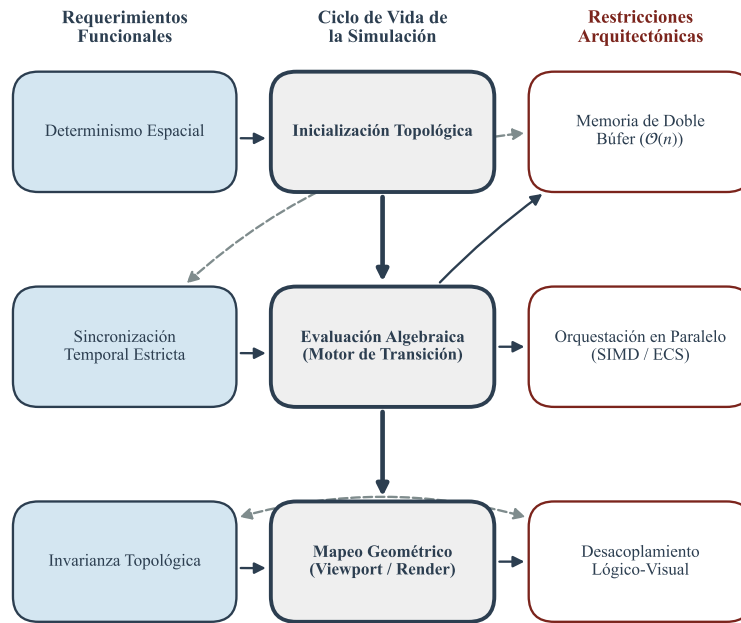


Fig. 6.1 Mapa conceptual de la interdependencia entre los requerimientos funcionales de simulación y las restricciones de diseño arquitectónico.

términos de control, pero estrictamente unidireccional en términos de dependencias lógicas, lo que facilita la mantenibilidad y la extensibilidad del motor.

Como se observa en la Figura 6.2, el sistema opera bajo un esquema de orquestación donde la capa superior gestiona el ciclo de vida, mientras que la capa inferior garantiza la ejecución determinista de la dinámica celular. Este diseño permite que múltiples instancias de simulación coexistan sin colisiones de memoria, cumpliendo así con el requerimiento funcional RF-01.

6.2.2 Taxonomía de las capas estructurales

A continuación, se describen conceptualmente las cuatro divisiones fundamentales que componen la topología del simulador:

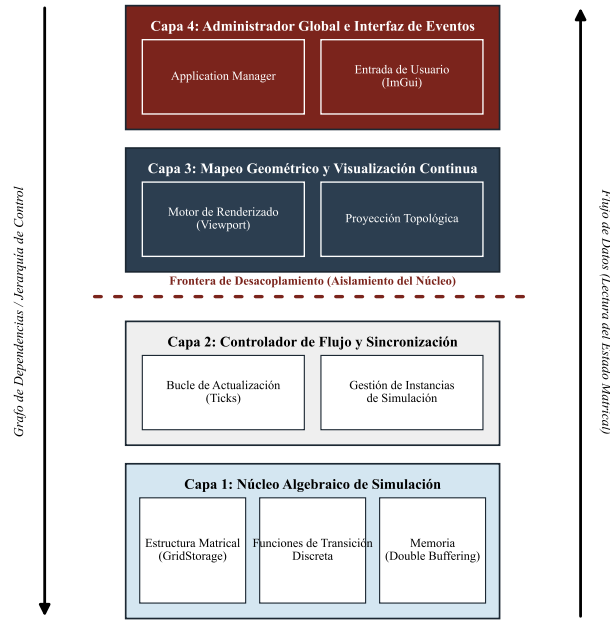


Fig. 6.2 Diagrama estructural de la arquitectura del sistema. Se ilustra la jerarquía de capas y el aislamiento del núcleo de simulación respecto a la interfaz de usuario.

1. **Núcleo de Simulación (Simulation Core):** Constituye la capa de abstracción de más bajo nivel y el corazón matemático del proyecto. Su responsabilidad es gestionar la representación del espacio celular $\mathcal{L}$ y la evaluación de la vecindad N . Implementa las estructuras de datos optimizadas (como `BitArray` y `GridStorage`) y ejecuta la lógica de actualización basada en la técnica de *double buffering* para garantizar el sincronismo perfecto exigido por la teoría formal [4].
2. **Control de Simulación (Simulation Control):** Esta capa actúa como un mediador entre el estado estático del autómata y su evolución dinámica. Gestiona el dominio temporal, controlando la velocidad de ejecución (pasos por segundo) y

la sincronización de los hilos de procesamiento en caso de existir concurrencia. Es la encargada de invocar el cálculo de la nueva generación basándose en los parámetros de la regla activa (e.g., *Spiral*).

3. **Representación y Visualización (Viewport System):** Se encarga de la traducción de los estados binarios almacenados en el núcleo hacia entidades geométricas renderizables. Implementa los isomorfismos espaciales demostrados en el Capítulo 4, transformando la lógica hexagonal en una representación visual coherente para el usuario. Esta capa es independiente de la simulación; es decir, el renderizado puede pausarse o modificarse sin alterar la evolución del sistema dinámico subyacente.
4. **Interfaz Gráfica y Administración Global (Application Layer):** Es el estrato de más alto nivel que coordina la aplicación en su totalidad. Gestiona el ciclo principal (*main loop*), la captura de eventos del teclado y ratón, y la administración de los paneles de configuración. Su función principal es la orquestación de los distintos *Viewports*, permitiendo la reconfiguración dinámica de las reglas en tiempo de ejecución (RF-02) mediante una interfaz no obstructiva.

Esta separación de responsabilidades no solo satisface los criterios de ingeniería de software, sino que proporciona un entorno controlado para validar las hipótesis de computabilidad universal planteadas en el Capítulo 1. Al garantizar que la visualización (Capa 3) es un reflejo fiel pero desacoplado de la lógica (Capa 1), los resultados experimentales sobre la colisión de partículas poseen un sustento técnico irrefutable.

6.3 Núcleo de simulación: Lógica fundamental y representación de estados

Una vez establecida la topología por capas en la Sección 6.2, es necesario profundizar en el diseño del *Simulation Core*. Este componente constituye la realización física del modelo matemático definido en la Sección 2.1, donde el autómata celular se concibe como una cuádrupla $(\mathcal{L}, S, N, \delta)$. La arquitectura del núcleo se centra en encapsular esta definición dentro de estructuras de datos que optimicen tanto el tiempo de acceso como el consumo de memoria, permitiendo la experimentación con las reglas complejas analizadas en el Capítulo 5.

6.3.1 Encapsulamiento del sistema dinámico

La entidad central del motor es la clase `Automaton`. Su función es integrar de manera coherente los elementos que rigen la evolución del sistema. A diferencia de implementaciones monolíticas tradicionales [7], esta clase actúa como un orquestador que vincula tres componentes fundamentales:

- **La Regla de Evolución (δ):** Encapsulada mediante un objeto de tipo *Rule*, el cual define la lógica de transición específica (e.g., *Spiral* o *Beehive*) sin alterar el resto del motor.
- **La Topología del Espacio (N):** Define la métrica de vecindad. En este proyecto, se implementa el mapeo submatricial de 2×2 demostrado en la Sección 4.3.2, permitiendo que una red ortogonal se comporte isomorficamente como una hexagonal.
- **El Almacenamiento Espacial ($\mathcal{L}$):** Gestionado por la clase *GridStorage*, la cual administra la memoria física de las celdas.

Esta centralización simplifica la reconstrucción del autómatas ante cambios de configuración. Al modificar parámetros como el tamaño de la malla o la regla de transición, la clase *Automaton* garantiza que los isomorfismos establecidos en el Capítulo 4 se mantengan consistentes, evitando estados inconsistentes durante la reconfiguración en tiempo de ejecución (RF-02).

6.3.2 Operaciones estructurales y evolutivas

El núcleo ejecuta operaciones críticas que permiten la transición del estado global de la red $C_t \rightarrow C_{t+1}$. La operación más relevante es el cómputo del paso de simulación, el cual debe realizarse de forma sincrónica para respetar la definición axiomática del sistema. En el Algoritmo 5 se describe este proceso, el cual integra la evaluación de la vecindad extendida definida en la Sección 4.4.1.

Algorithm 5 Cómputo de la transición sincrónica de la red.

```

UPDATESTEP( $\mathcal{G}, \delta$ )
  for each  $i, j$  in  $\mathcal{L}$ 
     $v \leftarrow \text{GETNEIGHBORHOOD}(\mathcal{G}, i, j)$ 
     $s' \leftarrow \delta(v)$ 
    SETNEXTSTATE( $\mathcal{G}, i, j, s'$ )
  SWAPBUFFERS( $\mathcal{G}$ )
return  $\mathcal{G}$ 

```

Además de la evolución temporal, el núcleo permite el redimensionamiento dinámico del espacio discreto y la modificación manual de estados (RF-03). Debido a la separación entre el almacenamiento y la lógica de transición, estas operaciones poseen una complejidad temporal de $O(1)$ para accesos individuales y $O(n)$ para transformaciones globales, alineándose con las cotas establecidas en la Sección 3.5.

6.3.3 Estructuras de almacenamiento discreto y optimización espacial

La gestión de memoria en el simulador trasciende la simple reserva de arreglos bidimensionales. Se emplea la clase `GridStorage` para implementar dos estrategias de optimización fundamentales para la viabilidad de la tesis:

6.3.3.1 Sustrato de doble almacenamiento (Double Buffering)

Para evitar las condiciones de carrera lógicas mencionadas en el Capítulo 3, el sistema mantiene dos búferes de memoria: el *Front Buffer* (lectura) y el *Back Buffer* (escritura). Como se fundamentó en la Sección 3.5, esta técnica duplica el requerimiento de memoria física para garantizar un sincronismo perfecto, permitiendo que la evaluación de δ dependa únicamente del estado global en el tiempo t [4].

6.3.3.2 Representación mediante arreglos de bits (BitArray)

Dado que los estados S de los autómatas complejos estudiados son binarios (0 o 1), el uso de tipos de datos estándar como `int` (32 bits) resultaría en un desperdicio del 96.8% del espacio. En su lugar, se implementa una estructura de tipo `BitArray`, donde cada celda ocupa exactamente un bit.

Representación	Espacio por Celda	Espacio para 10^6 celdas
Matriz estándar (<code>int</code>)	32 bits	≈ 4.0 MB
Matriz optimizada (<code>char</code>)	8 bits	≈ 1.0 MB
Estructura <code>BitArray</code>	1 bit	≈ 0.125 MB

Table 6.16 Comparativa de eficiencia espacial en la representación del estado celular.

Esta optimización reduce la complejidad espacial efectiva del sistema, facilitando la simulación de colisiones de gliders en mallas extensas que, de otro modo, presentarían cuellos de botella en la caché del procesador. Como se detalló en el análisis arquitectónico del sistema, esta decisión es el sustento para mantener una ejecución determinista sin comprometer el rendimiento de la aplicación activa.

6.4 Control de la dinámica temporal del autómata

Habiendo formalizado la encapsulación del espacio de estados $\mathcal{L}$ y la función local de transición δ dentro del núcleo de simulación (Sección 6.3), es imperativo definir el estrato arquitectónico responsable de la orquestación del tiempo. En la fundamentación teórica (Capítulo 2), la evolución temporal del autómata celular se concibe axiomáticamente como una progresión discreta $t \in \mathbb{N}$. Sin embargo, en el

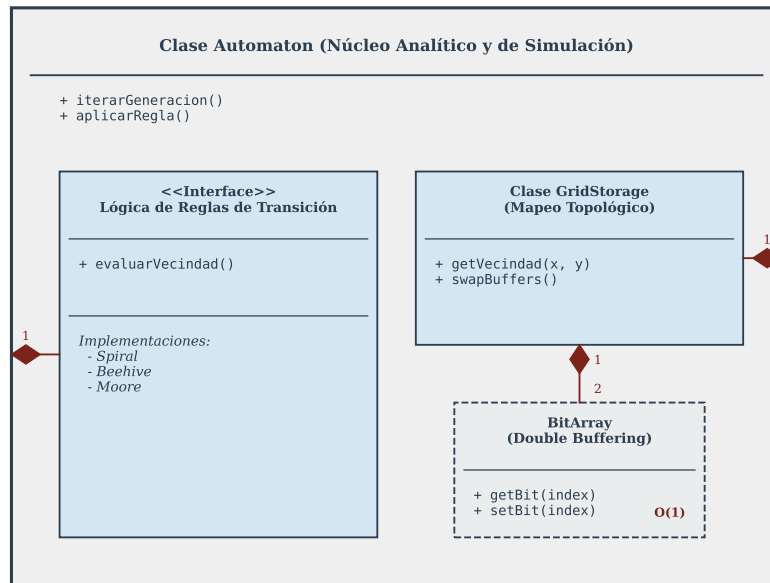


Fig. 6.3 Diagrama de encapsulamiento del núcleo. Se observa la agregación de la lógica de reglas y el almacenamiento optimizado dentro de la entidad Automaton.

dominio computacional, esta progresión lógica debe administrarse mecánicamente para sincronizarse con el hardware físico y la interacción del usuario.

Esta capa, posicionada jerárquicamente sobre el núcleo según la topología descrita en la Sección 6.2, aísla el comportamiento estático de las estructuras de memoria y dicta exclusivamente cuándo y cómo el sistema transita de una configuración C_t a C_{t+1} .

6.4.1 Delimitación entre el dominio lógico y el temporal

Para gestionar la dimensión cronológica, se diseña la clase *Simulation* como administrador operacional principal. La decisión de separar la entidad *Automaton* respecto a la entidad *Simulation* obedece a un principio fundamental de diseño de software: la separación de responsabilidades (*Separation of Concerns*).

Mientras que el *Automaton* sabe *cómo* evolucionar dadas las reglas topológicas analizadas en el Capítulo 4, carece de información sobre la frecuencia con la que debe hacerlo. Al instanciar la clase *Simulation* como un contenedor que envuelve al autómeta subyacente, se logra una ventaja arquitectónica crítica: el motor alge-

braico permanece puramente matemático y predecible, mientras que el controlador temporal maneja variables operacionales (como el tiempo transcurrido, las pausas y los incrementos manuales) sin contaminar el cálculo de la complejidad $O(n)$.

6.4.2 Administración de estados de ejecución y sincronización

La clase `Simulation` centraliza las operaciones de control de flujo, garantizando que los análisis dinámicos sobre configuraciones móviles (detallados en el Capítulo 5) puedan ser observados empíricamente de manera controlada. Las responsabilidades principales de este módulo incluyen:

- **Control de velocidad de simulación (TPS):** El sistema desacopla explícitamente los cuadros por segundo de la representación visual (*Frames Per Second*) de los pasos lógicos por segundo de la simulación (*Ticks Per Second* o TPS). Esto se implementa mediante un acumulador de tiempo que compensa las fluctuaciones del procesador, asegurando que el autómatas evolucione a una velocidad determinista fijada por el usuario, independientemente del rendimiento gráfico.
- **Máquina de estados de ejecución:** La orquestación temporal opera bajo una máquina de estados finitos que admite condiciones operativas como PAUSED, RUNNING y STEP. Esta estructura resulta indispensable para la síntesis heurística de circuitos (Sección 1.4), permitiendo pausar el universo, inyectar un *glider* manualmente y reanudar el cómputo sin perder el rastro generacional.
- **Randomización estocástica inicial:** El controlador expone rutinas para inyectar entropía en el espacio de estados. Proporciona la capacidad de sobrescribir el arreglo `BitArray` con una distribución uniforme de estados activos (1) e inactivos (0), estableciendo el punto de partida C_0 requerido para observar tiempos de estabilización y convergencia a atractores.

El proceso central que ejecuta el control temporal en cada iteración del bucle principal se formaliza en el Algoritmo 6. El uso de un acumulador evita la pérdida de iteraciones lógicas (*dropped ticks*) durante caídas de rendimiento del sistema.

Algorithm 6 Bucle de control temporal y orquestación del autómatas.

```

PROCESSTIMESTEP( $\mathcal{S}, \Delta t$ )
   $S_{acumulador} \leftarrow S_{acumulador} + \Delta t$ 
   $t_{paso} \leftarrow 1.0 / S_{tps\_objetivo}$ 
  while  $S_{acumulador} \geq t_{paso}$  do
    if  $S_{estado} = \text{RUNNING}$  then
      UPDATESTEP( $S_{automata}$ )
       $S_{generacion} \leftarrow S_{generacion} + 1$ 
    end if
     $S_{acumulador} \leftarrow S_{acumulador} - t_{paso}$ 
  end while
  return

```

Al mantener la semántica temporal aislada del renderizado visual y del estado lógico intrínseco, se establece un marco altamente robusto para experimentación. Cualquier alteración a la topología o a la regla de propagación (RF-02) no requiere modificación alguna en la lógica del temporizador expuesto en esta sección.

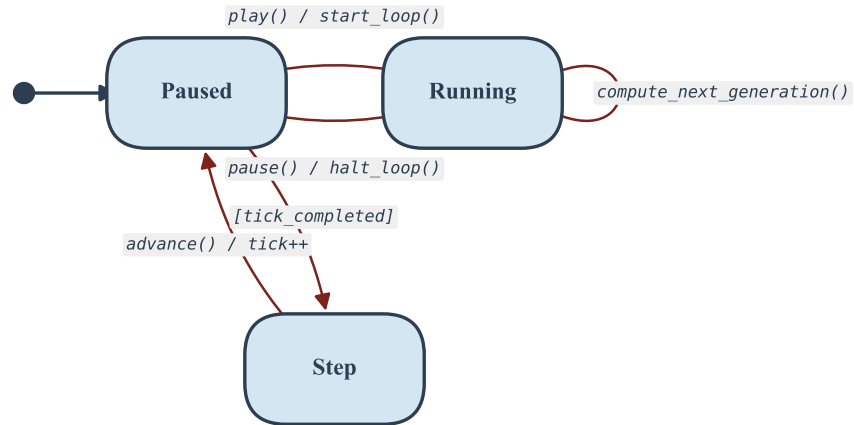


Fig. 6.4 Máquina de estados finitos que rige la clase *Simulation*, ilustrando las transiciones entre la pausa, la ejecución continua y el avance paso a paso controlado.

6.5 Aislamiento espacial y multiplicidad de entornos (Sistema Multi-Viewport)

La validación empírica del isomorfismo topológico definido en el Capítulo 4 exige un entorno capaz de contrastar distintos modelos geométricos y dinámicos de manera concurrente. Para satisfacer el requerimiento funcional de instanciación múltiple (RF-01, Tabla 6.1), la arquitectura del sistema implementa un modelo de aislamiento espacial denominado *Multi-Viewport*.

Este modelo trasciende la capacidad de una simple interfaz gráfica al establecer contenedores lógicos que agrupan y aíslan el núcleo matemático (Sección 6.3) y

el control temporal (Sección 6.4). En consecuencia, se garantiza la coexistencia paralela de diversos universos celulares sin incurrir en colisiones de memoria ni en condiciones de carrera durante la evaluación de la función de transición discreta.

6.5.1 Instanciación independiente de contextos evolutivos

En términos arquitectónicos, un *Viewport* se define como una entidad superior que representa una instancia completamente independiente de simulación y visualización. Cada objeto *Viewport* posee un ciclo de vida encapsulado y mantiene referencias exclusivas hacia sus propias instancias de las clases *Automaton* y *Simulation*.

Esta delimitación estructural permite evaluar el Teorema de equivalencia computacional (Sección 4.5.2) empíricamente. Por ejemplo, es posible inicializar un *Viewport A* configurado con una topología estrictamente ortogonal y una regla de propagación clásica, mientras simultáneamente un *Viewport B* ejecuta el mapeo submatricial direccional regido por la dinámica *Spiral*. Debido a que la memoria de red (*GridStorage*) de cada autómatata reside en espacios de direcciones distintos, el aislamiento es absoluto; el comportamiento divergente o convergente de los atractores puede observarse en tiempo real sin perturbaciones cruzadas.

6.5.2 Composición y orquestación del entorno aislado

Para proporcionar una interfaz de observación coherente sobre el sustrato celular mapeado, cada *Viewport* requiere administrar componentes operativos locales. Esta composición interna se divide fundamentalmente en cuatro módulos:

1. **Cámaras independientes:** Cada entorno gestiona su propia matriz de proyección y transformación afín sobre el espacio continuo $\mathbb{R}^2$. Esto permite operaciones de traslación y acercamiento (*pan* y *zoom*) locales, facilitando el seguimiento de configuraciones móviles (*gliders*) en un universo sin afectar la vista del resto.
2. **Layout espacial:** Define los límites de renderizado (*bounding box*) del entorno dentro de la ventana global de la aplicación.
3. **Configuración de renderizado:** Mantiene descriptores visuales específicos, tales como las paletas de colores asociadas a los estados binarios del alfabeto Σ y la habilitación de rejillas delimitadoras.
4. **Manejadores de interacción:** Subrutinas encargadas de interceptar los eventos periféricos (clics de ratón) y traducirlos hacia el dominio discreto.

La orquestación de interacciones es particularmente crítica cuando coexisten múltiples topologías. Cuando el usuario interactúa con la pantalla, el sistema debe determinar qué universo celular recibe el estímulo. Posteriormente, aplica el mapeo espacial inverso analizado en la Sección 3.5.1 para resolver la coordenada continua

(x, y) hacia su correspondiente índice matricial discreto (i, j) . Este mecanismo de despacho (RF-03) se describe formalmente en el Algoritmo 7.

Algorithm 7 Despacho global de interacciones de usuario en entornos Multi-Viewport.

```

HANDLEINPUT( $\mathcal{V}_{list}, coord\_x, coord\_y, estado\_click$ )
  for each  $v$  in  $\mathcal{V}_{list}$ 
    if ISPOINTINSIDEBOUNDS( $v, coord\_x, coord\_y$ ) then
       $\Delta x, \Delta y \leftarrow \text{GETCAMERAOFFSET}(v)$ 
       $x_{local} \leftarrow coord\_x - \Delta x$ 
       $y_{local} \leftarrow coord\_y - \Delta y$ 
       $i, j \leftarrow \text{INVERSESPATIALMAPPING}(v.topologia, x_{local}, y_{local})$ 
      if  $i, j$  in  $v.automata.\mathcal{L}$  then
        SETNEXTSTATE( $v.automata, i, j, estado\_click$ )
      end if
    end if
  end for
  return

```

Este algoritmo iterativo de complejidad espacial $O(1)$ y temporal $O(k)$ (donde k es el número de viewports activos) asegura que la alteración manual del autómata no interrumpa el hilo de simulación subyacente. La incorporación del sistema *Multi-Viewport* consolida al software como un laboratorio algorítmico, proporcionando la infraestructura modular requerida para sintetizar y depurar los circuitos lógicos propuestos en el Capítulo 5.

6.6 Mapeo geométrico y representación visual discreta

Una vez garantizado el aislamiento espacial de los entornos en la Sección 6.5, es necesario formalizar el mecanismo de proyección que traduce las estructuras matemáticas subyacentes hacia entidades gráficas comprensibles por el ojo humano. La transición desde el estado lógico $s \in \Sigma$ hacia una representación visual tangible no es una operación trivial, pues implica la aplicación de las funciones de transformación afín y los isomorfismos espaciales desarrollados en el Capítulo 3.

En esta capa de la arquitectura, el sistema debe procesar el arreglo de bits (*BitArray*) y, basándose en la topología activa, generar la geometría correspondiente en el espacio continuo $\mathbb{R}^2$. Este proceso constituye la realización visual del Teorema de equivalencia computacional (Sección 4.5.2), permitiendo validar que la dinámica de partículas se comporta según lo previsto por la teoría.

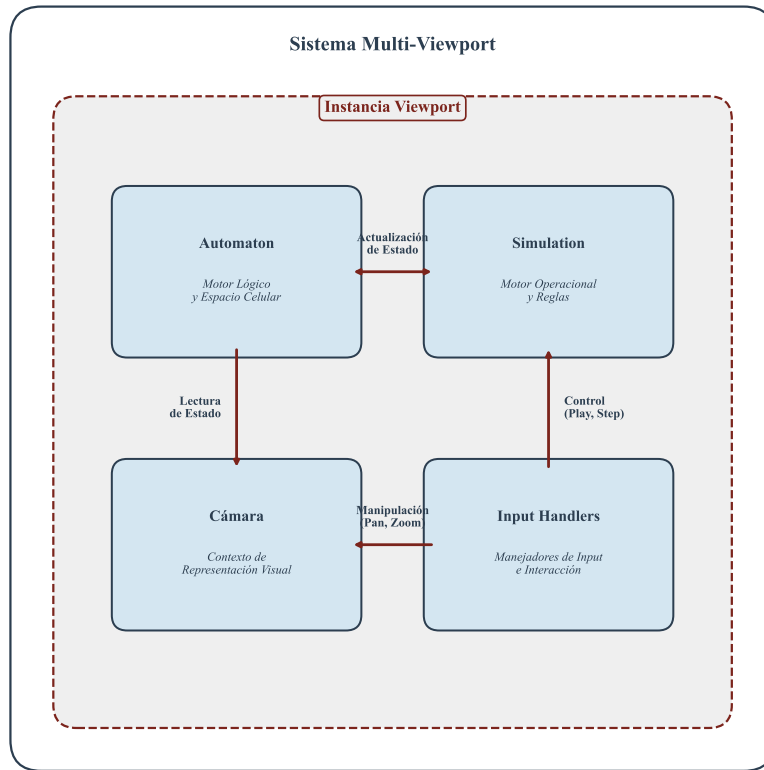


Fig. 6.5 Estructura interna del sistema Multi-Viewport. Se muestra el encapsulamiento de los motores lógicos y operacionales bajo un mismo contexto de representación visual.

6.6.1 Desacoplamiento de la geometría y el estado lógico

El diseño del simulador prioriza un desacoplamiento estricto entre la evaluación de la función de transición δ y su dibujo en pantalla. Para lograr este objetivo, se introduce la interfaz abstracta **AutomatonRenderer**. Esta estructura actúa como un contrato de software que define las operaciones necesarias para visualizar un autómata, sin conocer los detalles de su topología interna.

La ventaja técnica de este enfoque radica en la extensibilidad: es posible incorporar múltiples tipos de renderizadores (e.g., ortogonales, hexagonales, o incluso representaciones tridimensionales) y alterar la estética de la simulación sin contaminar ni comprometer la lógica de cómputo descrita en la Sección 6.3. Este diseño asegura que el motor de simulación permanezca como un artefacto puramente matemático, mientras que el renderizador opera como un observador pasivo.

6.6.2 Implementaciones de transformación espacial

Las especializaciones de la interfaz `AutomatonRenderer` son las responsables de aplicar las ecuaciones de mapeo direccional definidas en la Ecuación (4.15). A continuación, se describen las responsabilidades de estas especializaciones, tomando como referencia el renderizado hexagonal:

- **Generación de mallas dinámicas:** El renderizador calcula la posición de los vértices de cada celda basándose en el radio r y el *offset* de fase definido en la Sección 3.3.
- **Mapeo de estados a descriptores visuales:** Traduce el valor binario del estado (0 o 1) hacia propiedades de color y opacidad. En los experimentos de colisión de partículas (Capítulo 5), esto permite diferenciar claramente los frentes de onda de las estructuras estables (*still lifes*).
- **Optimización del ciclo de dibujo:** Dado que el número de celdas n puede ser elevado, el renderizador implementa técnicas de visibilidad para dibujar únicamente las celdas contenidas dentro del área de recorte (*frustum*) del Viewport activo.

El proceso de renderizado iterativo se formaliza en el Algoritmo 8. Es fundamental observar que este algoritmo posee una complejidad temporal $O(n)$, alineada con el análisis asintótico del Capítulo 3.

Algorithm 8 Proyección visual del espacio de estados discretos.

```

RENDERAUTOMATON( $\mathcal{G}, \mathcal{R}, C$ )
  for each  $i, j$  in  $\mathcal{G}.\mathcal{L}$ 
     $s \leftarrow \text{GETCELLSTATE}(\mathcal{G}, i, j)$ 
    if  $s$  is active then
       $x, y \leftarrow \text{CALCULATESPATIALPOSITION}(i, j, \mathcal{G}.\text{topologia})$ 
      if  $\text{ISVISIBLE}(x, y, C)$  then
         $\text{DRAWHEXAGONSHAPE}(x, y, \mathcal{R}.\text{color\_activo})$ 
      end if
    end if
  end for
return
```

Al finalizar este proceso, la malla ortogonal de memoria se percibe visualmente como una teselación hexagonal perfecta, cumpliendo con el requerimiento funcional de fidelidad topológica (RF-04). Esta representación visual es el sustento para la captura de los datos estadísticos y la observación de los atractores en las reglas *Spiral* y *Beehive*, ya que permite al investigador identificar patrones emergentes que en una matriz de datos crudos serían imperceptibles [7].

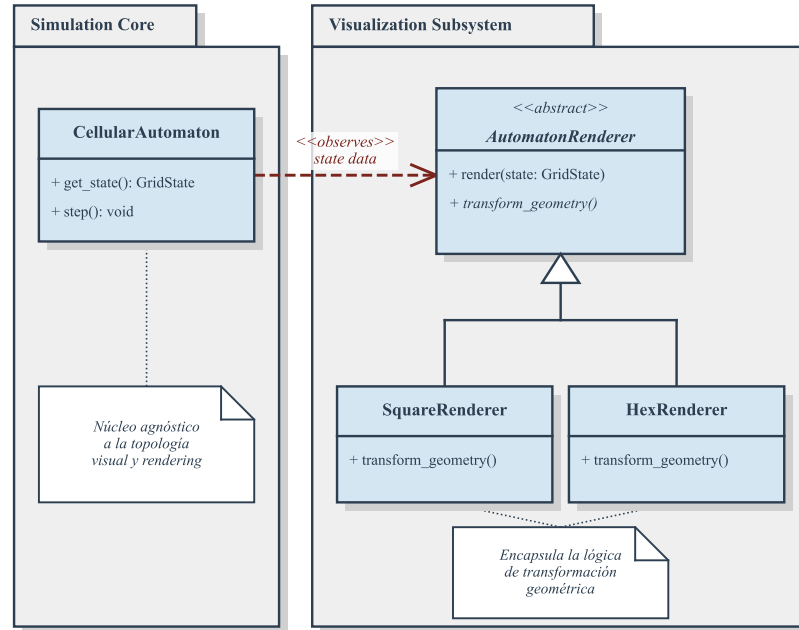


Fig. 6.6 Arquitectura del subsistema de visualización. Se ilustra cómo las especializaciones de renderizado encapsulan la lógica de transformación geométrica sin afectar al núcleo de simulación.

6.7 Orquestación global e interacción asíncrona del sistema

Tras haber definido los componentes de renderizado y el aislamiento espacial en las Secciones 6.5 y 6.6, es imperativo establecer el mecanismo que coordina la coexistencia de estos módulos. La capa superior de la arquitectura es la encargada de administrar el ciclo de vida de la aplicación y procesar los estímulos del usuario de manera no obstructiva. En este nivel, la abstracción matemática se integra con la ingeniería de interfaces para proporcionar un entorno que permita la experimentación con los circuitos lógicos definidos en la Sección 1.4.

6.7.1 Coordinación del ciclo principal y sincronización de estados

El coordinador central del sistema se materializa en la clase `Application`. Esta entidad posee la responsabilidad única de gestionar el bucle principal (*Main Loop*), el cual rige la cadencia de ejecución del simulador. A diferencia de arquitecturas

basadas estrictamente en la delegación por eventos, se optó por un modelo de sincronización explícita de estados. Esta decisión técnica se fundamenta en la necesidad de reducir la complejidad accidental del software, garantizando que el flujo de información sea predecible y que la simulación no se vea afectada por latencias en el subsistema de interfaz.

Las responsabilidades de la clase **Application** incluyen:

- **Gestión del ciclo de vida:** Controla la inicialización de los contextos gráficos y la terminación segura de los hilos de simulación.
- **Procesamiento de eventos globales:** Intercepta las señales del sistema operativo (ventana, teclado y ratón) para despacharlas hacia el *Viewport* activo mediante el Algoritmo 7.
- **Arbitraje de instancias:** Administra la lista de entornos evolutivos, permitiendo al usuario alternar entre simulaciones con distintas reglas (e.g., *Spiral* y *Beehive*) sin interrumpir la dinámica de fondo.

El funcionamiento de este coordinador se formaliza en el Algoritmo 9, donde se observa la secuencia atómica de actualización y dibujo que asegura la consistencia del sistema.

Algorithm 9 Bucle principal de orquestación del simulador.

```

RUNAPPLICATION( $\mathcal{A}$ )
  while  $\mathcal{A}.activo$  do
     $\Delta t \leftarrow \text{GetFRAMETime}()$ 
    POLLEVENTS()
    UPDATEINTERFACE(ImGui)
    for each  $v$  in  $\mathcal{A}.\mathcal{V}_{list}$ 
      PROCESSTIMESTEP( $v, \Delta t$ )
    end for
    BEGINFRAME()
    RENDERALLVIEWPORTS( $\mathcal{A}.\mathcal{V}_{list}$ )
    RENDERINTERFACEPANELS()
    ENDFRAME()
  end while
return
```

6.7.2 Organización modular de la interfaz de usuario

La interacción con el autómata celular complejo requiere de herramientas de monitoreo en tiempo real. La arquitectura de la interfaz se fragmenta en paneles independientes implementados mediante la biblioteca *ImGui*, permitiendo un flujo de trabajo asíncrono. Estos paneles se especializan en tareas críticas como la visualización de estadísticas generacionales, la configuración de la apariencia y la graficación de la entropía del sistema analizada en el Capítulo 2.

Para optimizar el área de trabajo, la interfaz evolucionó hacia un modelo de contenedores acoplables (*Docking*), donde cada panel puede ser reubicado según las necesidades del experimento. La comunicación entre los paneles y el simulador se realiza mediante el uso de referencias no propietarias (*raw pointers* o *weak references*) hacia el *Viewport* activo. Este patrón de diseño asegura que la interfaz pueda consultar y modificar el estado del autómata (RF-03) sin romper el encapsulamiento del núcleo algebraico, manteniendo la integridad de las estructuras *BitArray* descritas en la Sección 6.3.

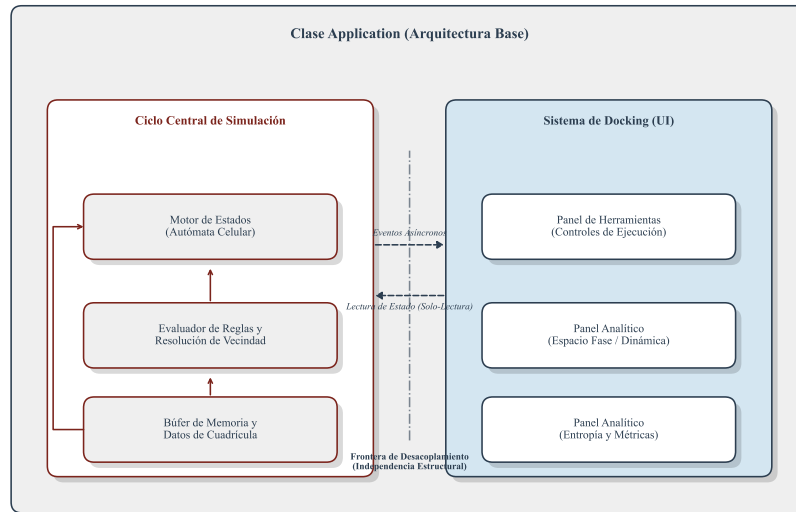


Fig. 6.7 Modelo estructural de la interfaz de usuario. Se ilustra la independencia de los paneles analíticos respecto al ciclo de simulación central.

Esta estructura modular permite que el simulador funcione como una plataforma extensible. La adición de nuevas herramientas de análisis, como inspectores de vecindad o pinceles de patrones, se realiza de forma desacoplada, cumpliendo con los atributos de mantenibilidad y modularidad (RNF-02) establecidos al inicio de este capítulo.

6.8 Consecuencias arquitectónicas y proyecciones futuras

La culminación del diseño del motor de simulación permite realizar un balance analítico sobre la convergencia entre la teoría formal de autómatas celulares y la ingeniería de software. Como se estableció en los objetivos sintéticos (Sección 1.4), el

sistema debe operar como un sustrato determinista para la validación de isomorfismos topológicos. La arquitectura resultante no es solo un entorno de ejecución, sino un marco de trabajo modular que garantiza que la dinámica observada en las reglas *Spiral* y *Beehive* sea una consecuencia directa de las funciones de transición δ y no un artefacto de la implementación física.

6.8.1 Evaluación de la modularidad y robustez del sistema

Al contrastar la arquitectura actual con las implementaciones monolíticas iniciales [7], se identifica que el desacoplamiento estratificado ha incrementado la robustez del modelo. La separación en capas (Sección 6.2) permite que el núcleo algebraico permanezca invariable ante alteraciones en la interfaz de usuario o en el subsistema de renderizado.

Esta flexibilidad se manifiesta en la capacidad del motor para integrar nuevas topologías de red sin modificar las subrutinas de control temporal. Gracias al uso de la interfaz abstracta `AutomatonRenderer` (Sección 6.6), la incorporación de una malla asimétrica o una vecindad de radio $r > 1$ se reduce a la especialización de funciones de mapeo espacial, preservando la integridad del bucle principal gestionado por la clase `Application`. En términos de ingeniería, se ha logrado que el sistema cumpla con el requerimiento RNF-02 (Tabla 6.12), minimizando la complejidad accidental y facilitando la mantenibilidad a largo plazo.

6.8.2 Líneas de refinamiento y herramientas analíticas desacopladas

A pesar de la solidez estructural alcanzada, se identifican áreas donde la subdivisión de responsabilidades puede profundizarse para mejorar la experiencia analítica. Actualmente, la entidad *Viewport* (Sección 6.5) centraliza tanto la interacción periférica como la administración de cámaras; una evolución prospectiva sugiere la fragmentación de estos roles en módulos de interacción dedicados.

Asimismo, se propone el desarrollo de las siguientes capacidades operativas:

- **Sistemas de serialización y persistencia:** Implementar esquemas de guardado en formatos estándar (e.g., JSON o binario crudo) para capturar estados globales C_t . Esto permitiría la replicación exacta de colisiones de *gliders* en diferentes sesiones de investigación.
- **Herramientas de edición desacopladas:** Evolucionar hacia un modelo de herramientas basadas en pinceles (*Brushes*) y herramientas de selección. Esto facilitaría la síntesis heurística de circuitos complejos descrita en el Capítulo 5, permitiendo la inserción atómica de patrones predefinidos (atractores y osciladores).

6.8.3 Optimización asintótica y arquitecturas orientadas a datos

Para trascender los límites de simulación en mallas extensas, donde el número de celdas $n \rightarrow \infty$, es imperativo considerar optimizaciones que aprovechen las arquitecturas de cómputo moderno. Aunque el uso de `BitArray` (Sección 6.3) optimiza la huella de memoria, la evaluación secuencial de δ en una CPU unihilo presenta un cuello de botella latente.

En este sentido, se proponen las siguientes rutas de optimización técnica:

1. **Paralelización computacional:** Dado que la actualización de un autómata celular es un proceso inherentemente paralelo (cada celda depende únicamente del estado anterior de su vecindad), la función `UPDATESTEP` puede ser distribuida en múltiples hilos mediante librerías de concurrencia.
2. **Instrucciones SIMD (Single Instruction, Multiple Data):** El uso de vectores de procesamiento a nivel de registro permitiría evaluar múltiples celdas del `BitArray` en un solo ciclo de instrucción, reduciendo significativamente el tiempo de computación $T(n)$.
3. **Arquitectura ECS (Entity Component System):** Para la gestión visual de frentes de onda y partículas masivas, una transición hacia un modelo orientado a datos permitiría manejar las subconfiguraciones móviles como entidades independientes, optimizando el uso de la caché del procesador.

El Algoritmo 10 ilustra conceptualmente cómo se estructuraría una actualización paralela dividida en fragmentos espaciales (*chunks*), manteniendo la consistencia del *double buffering*.

Algorithm 10 Estructura conceptual de evolución paralela por fragmentación.

```

PARALLELUPDATE( $\mathcal{G}$ ,  $\delta$ )
   $C_{list} \leftarrow \text{SUBDIVIDE\_LATTICE}(\mathcal{G}, \mathcal{L})$ 
  parallel for each  $c$  in  $C_{list}$ 
    UPDATECHUNKSTEP( $\mathcal{G}$ ,  $c$ ,  $\delta$ )
  end parallel
  SWAPBUFFERS( $\mathcal{G}$ )
return

```

Estas proyecciones aseguran que el simulador no solo resuelva las preguntas de investigación del presente Trabajo Terminal, sino que se posicione como una herramienta base para el estudio de la computabilidad universal en sistemas dinámicos discretos de alta complejidad [5].

Referencias

1. J. von Neumann y A. W. Burks, *Theory of Self-Reproducing Automata*. Urbana, IL, USA: University of Illinois Press, 1966.
2. S. Wolfram, *A New Kind of Science*. Champaign, IL, USA: Wolfram Media, 2002.
3. A. Wuensche, "Genomic regulation, pattern formation and spatial chaos in continuous media," en *Artificial Life VI: Proc. of the Sixth Int. Conf. on Artificial Life*, Cambridge, MA, USA: MIT Press, 1998, pp. 246–254.
4. T. Toffoli y N. Margolus, *Cellular Automata Machines: A New Environment for Modeling*. Cambridge, MA, USA: MIT Press, 1987.
5. J. Kari, "Theory of cellular automata: A survey," *Theoretical Computer Science*, vol. 334, no. 1-3, pp. 3–33, 2005.
6. A. Adamatzky, *Identification of Cellular Automata*. Londres, Reino Unido: Taylor & Francis, 1994.
7. R. Basurto Flores y P. A. León Hernández, "Computación basada en reacción de partículas en un autómata celular hexagonal," Trabajo Terminal No. 20080040, Escuela Superior de Cómputo, Instituto Politécnico Nacional, México, 2009.
8. T. H. Cormen, C. E. Leiserson, R. L. Rivest, y C. Stein, *Introduction to Algorithms*, 3.^a ed. Cambridge, MA, USA: MIT Press, 2009.
9. S. I. Grossman, *Álgebra Lineal*, 7ma ed. México: McGraw-Hill Interamericana, 2012.